E-COMMERCE · ANALYTICS · QUERY TUNING

종합 실습 4
ecommerce 매출 분석 및 정리

전자상거래 주문·상품·고객·재고·리뷰 데이터를 대상으로 매출, RFM, 재구매율, 재고 위험, 쿠폰 효과를 분석하고 실행 계획과 인덱스를 이용해 성능을 비교했다.

소속	이름	DBMS
광주 3반	박상명	PostgreSQL

목차

1. 10개의 문항별 튜닝 전 쿼리와 결과 화면, 튜닝 후 쿼리와 결과 화면 정리
2. 3가지 조인에 대한 차이점 정리 (Hash Join / Nested Loop / Bitmap Heap Scan)
3. Materialized View 생성 스크립트와 실행 후 결과 화면
4. DBMS 엔진별 옵티마이저 조사

1. 10개 문항별 튜닝 전·후 쿼리와 결과 화면

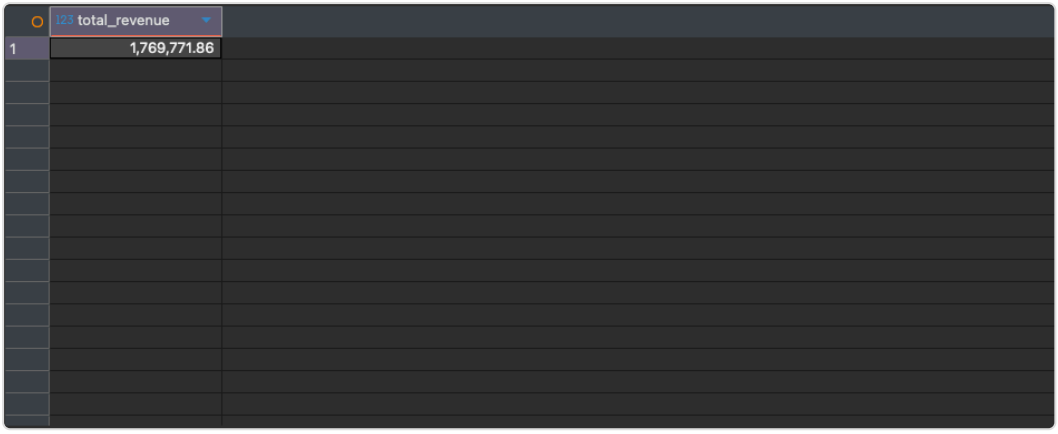
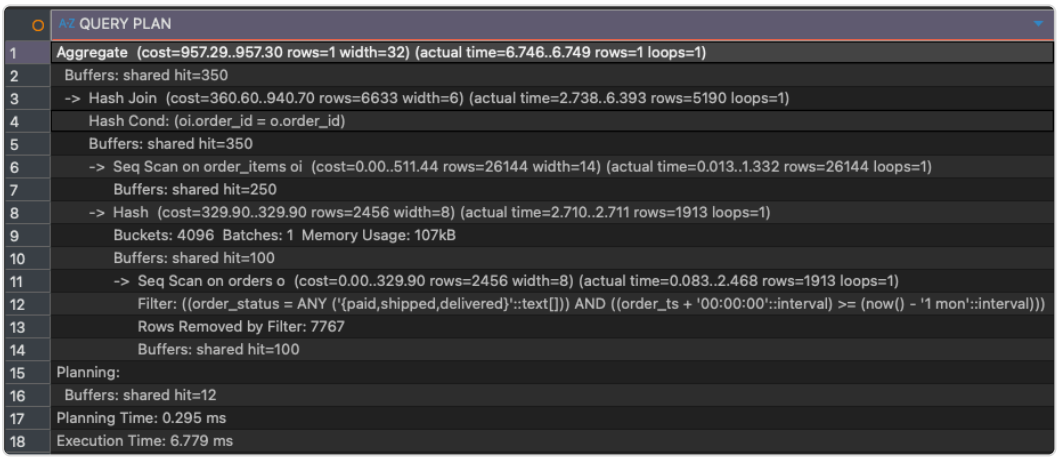
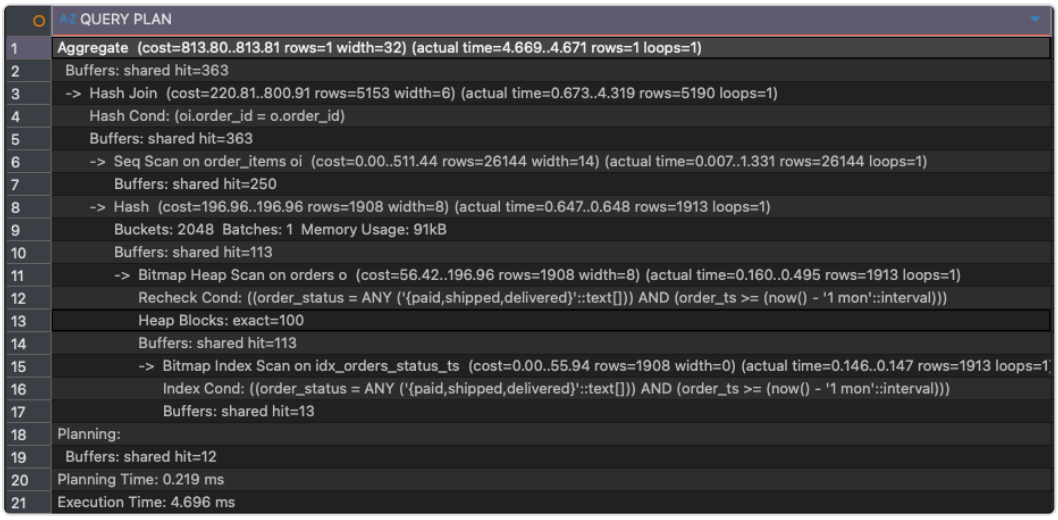
Q1) 지난 한 달간 실제 팔린 총 금액 (paid+shipped+delivered)

쿼리 의도: `orders` 와 `order_items` 를 조인해 최근 1개월간 paid/shipped/delivered 상태인 주문의 매출 합계를 구한다.

튜닝 전 문제점 `WHERE (o.order_ts + INTERVAL '0 seconds') >= now() - INTERVAL '1 month'` 처럼 인덱스 대상 컬럼(`order_ts`)에 연산을 걸어 비교했다. 컬럼에 연산이 붙으면 옵티마이저가 인덱스의 Index Cond로 활용하지 못하고 Seq Scan 후 Filter로 처리하게 된다.

튜닝 내용 - `CREATE INDEX idx_orders_status_ts ON orders (order_status, order_ts)` 복합 인덱스 추가 - WHERE 절을 `o.order_ts >= now() - INTERVAL '1 month'` 로 되돌려 컬럼을 연산 없이 원본 그대로 비교

효과: `order_status` 등치 조건과 `order_ts` 범위 조건이 함께 Index Cond로 결합되어 Index Scan(혹은 Bitmap Index Scan)으로 전환, 스캔 대상 행 수가 크게 감소.

구분	스크린샷
결과 확인 (튜닝 전/후 공통)	
튜닝 전 EXPLAIN ANALYZE	
튜닝 후 EXPLAIN ANALYZE	

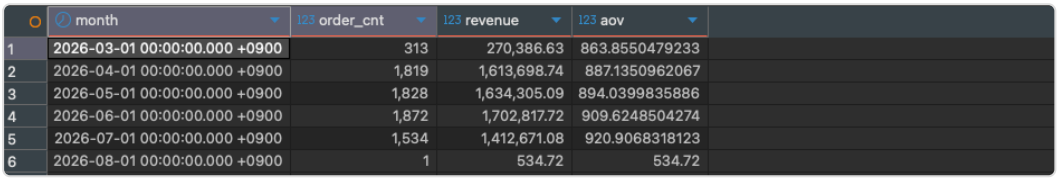
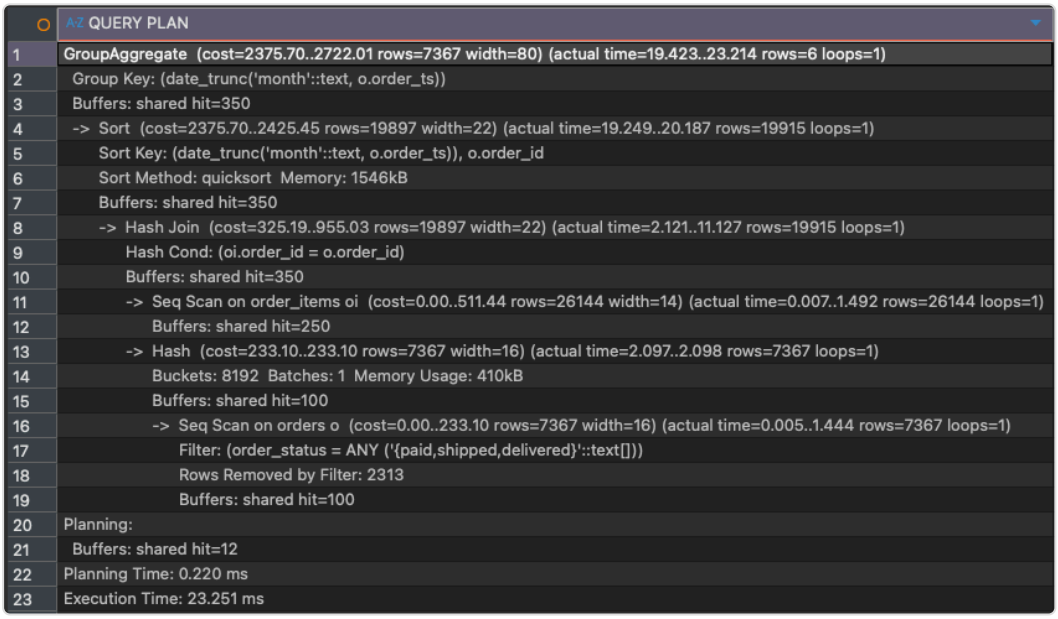
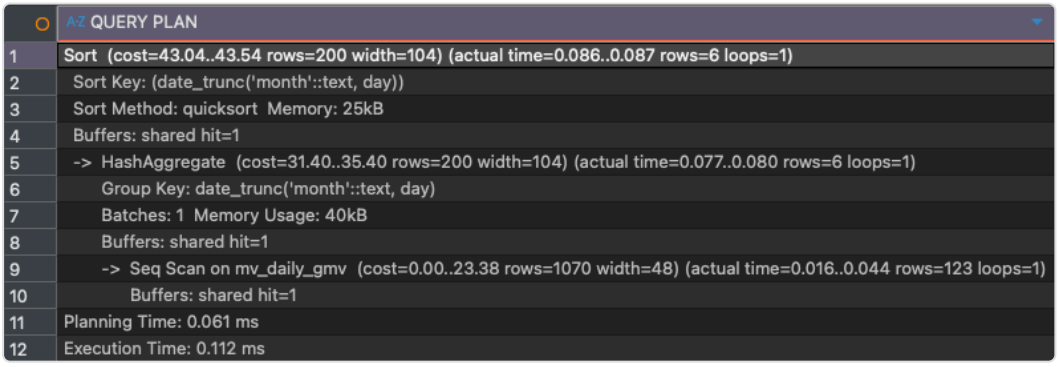
Q2) 월별 주문수 / 매출 / 평균주문금액 (AOV)

쿼리 의도: 월 단위로 주문 건수, 총 매출, 주문당 평균 금액(AOV)을 집계하는 리포트 쿼리.

튜닝 전 문제점 리포트성 조회임에도 조회할 때마다 `orders ⋈ order_items` 전체를 조인하고 `GROUP BY` 로 집계한다. 원본 테이블이 커질수록 매번 무거운 조인·집계 비용을 반복해서 지불하게 된다.

튜닝 내용 - `mv_daily_gmv` Materialized View를 일별 `order_cnt` / `gmv` 까지 포함하도록 재정의 - `idx_mv_daily_gmv_day` 인덱스 추가 - 조회 쿼리는 원본 테이블 조인 없이 `mv_daily_gmv` 만 스캔해서 월 단위로 재집계(`SUM(order_cnt)` , `SUM(gmv)`)

효과: 무거운 조인·집계 비용을 MV 갱신(refresh) 시점에 미리 지불해두고, 반복되는 리포트 조회는 훨씬 작은 사전집계 테이블만 스캔하면 되므로 응답 속도가 크게 개선된다. (단, MV는 refresh 전까지는 최신 데이터를 반영하지 못하므로 최신성과 조회속도 사이의 트레이드오프가 존재 — 갱신 주기 설계가 필요)

구분	스크린샷
결과 확인 (튜닝 전/후 공통)	
튜닝 전 EXPLAIN ANALYZE	
튜닝 후 EXPLAIN ANALYZE	

Q3) 최근 90일 카테고리 Top10

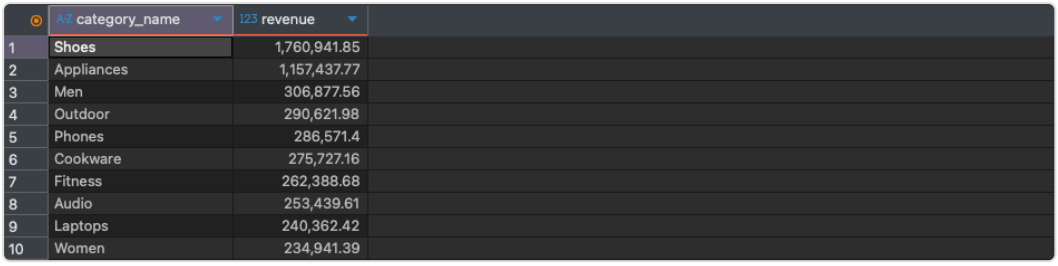
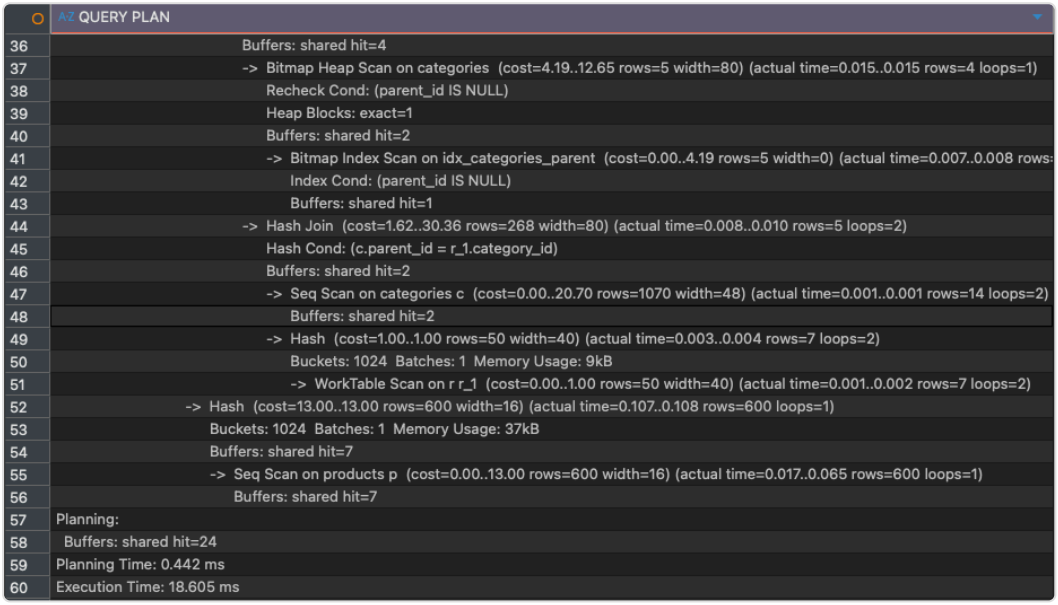
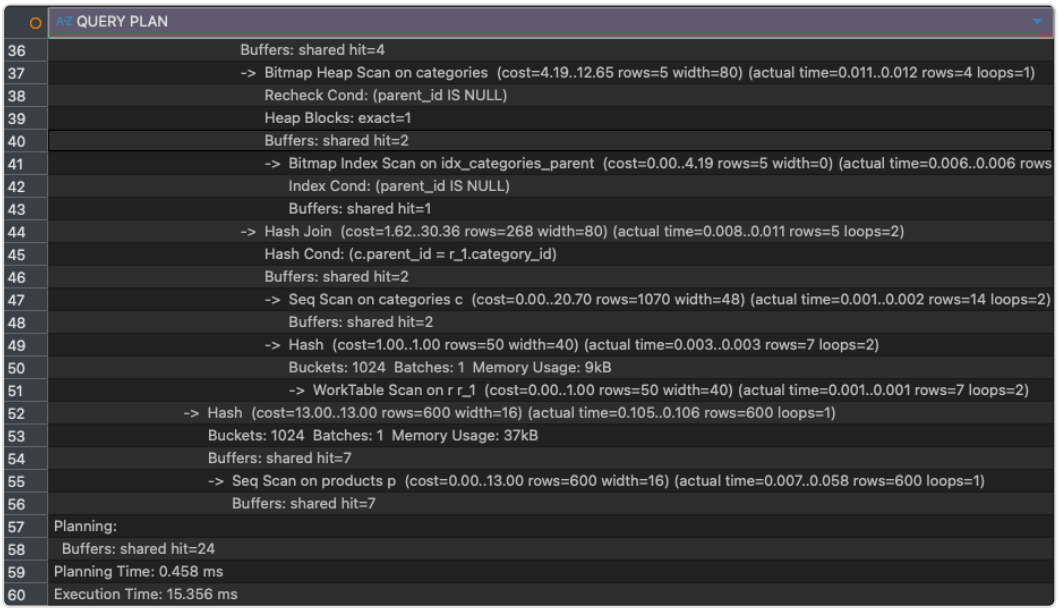
쿼리 의도: 최근 90일간 카테고리별 매출을 집계해 상위 10개 카테고리를 조회.

튜닝 전 문제점 Q1과 동일하게 `(o.order_ts + INTERVAL '0 seconds') >= now() - INTERVAL '90 days'` 형태로 컬럼에 연산을 걸어 인덱스를 활용하지 못한다.

튜닝 내용 - 별도 인덱스를 새로 만들지 않고, Q1에서 생성한 `idx_orders_status_ts`를 그대로 재사용 - WHERE 절 비교를 `o.order_ts >= now() - INTERVAL '90 days'`로 원본 컬럼 비교로 되돌림

효과: 여러 쿼리가 동일한 필터 패턴(`order_status` + `order_ts` 범위)을 공유한다면, 인덱스 하나로 여러 쿼리를 함께 커버할 수 있음을 보여준다 — 인덱스 설계 시 재사용성을 고려해야 하는 사례.

참고: 현재 카테고리 트리는 리프 카테고리가 10개뿐이라 Top10 결과가 전체 카테고리와 같을 수 있음(정상).

구분	스크린샷
결과 확인 (튜닝 전/후 공통)	
튜닝 전 EXPLAIN ANALYZE	
튜닝 후 EXPLAIN ANALYZE	

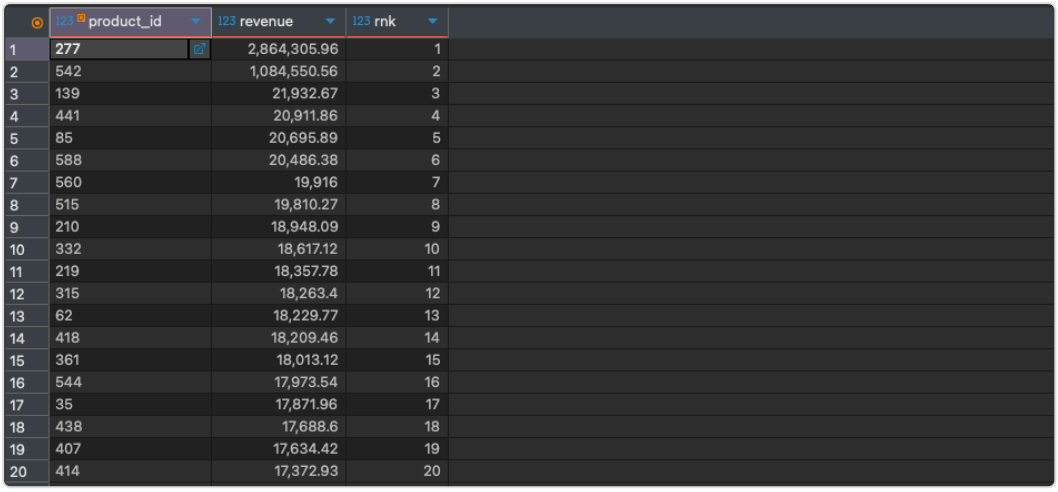
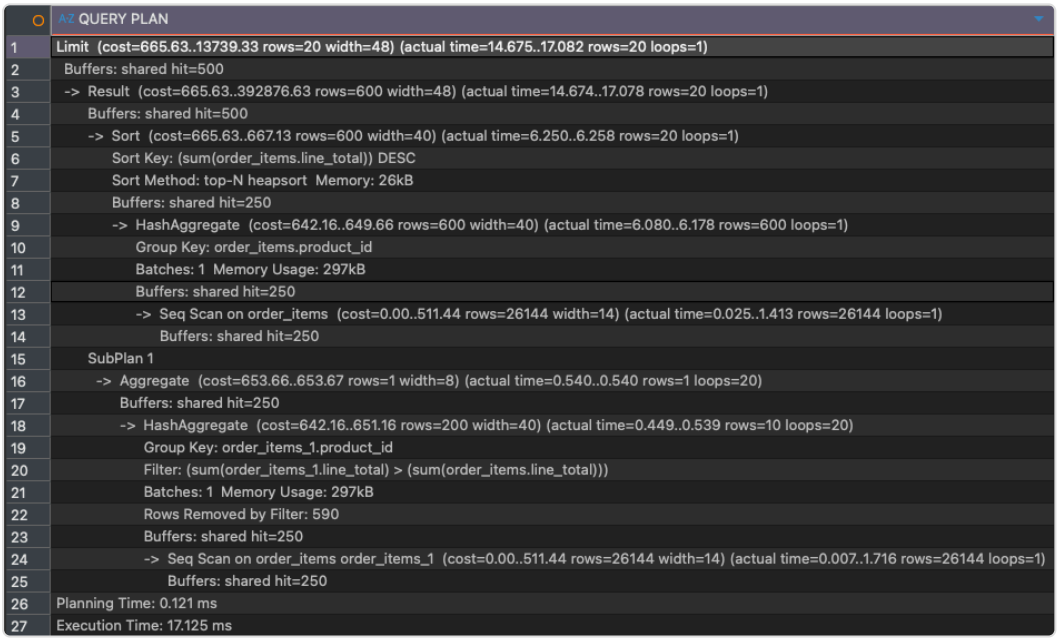
Q4) 제품별 누적매출 RANK() Top20

쿼리 의도: 제품별 누적 매출을 구하고, 매출 기준 순위를 매겨 상위 20개 제품을 조회.

튜닝 전 문제점 "나보다 매출이 큰 제품 수 + 1"을 구하는 상관 서브쿼리(correlated subquery)로 순위를 흉내 냈다. 제품마다 전체 제품 집계 결과를 다시 스캔하므로, 제품 수가 N개면 사실상 $O(N^2)$에 가까운 반복 스캔이 발생한다.

튜닝 내용 - `RANK() OVER (ORDER BY SUM(line_total) DESC)` 윈도우 함수로 대체해 단일 스캔+정렬로 순위를 계산 - `CREATE INDEX idx_order_items_product_cover ON order_items (product_id) INCLUDE (line_total)` 커버링 인덱스 추가

효과: $O(N^2)$ 상관 서브쿼리 → 단일 스캔·정렬($O(N \log N)$)로 전환되고, 커버링 인덱스 덕분에 Index Only Scan이 가능해져 힙 접근(Heap Fetch)이 제거된다.

구분	스크린샷
결과 확인 (튜닝 전/후 공통)	
튜닝 전 EXPLAIN ANALYZE	
튜닝 후 EXPLAIN ANALYZE	

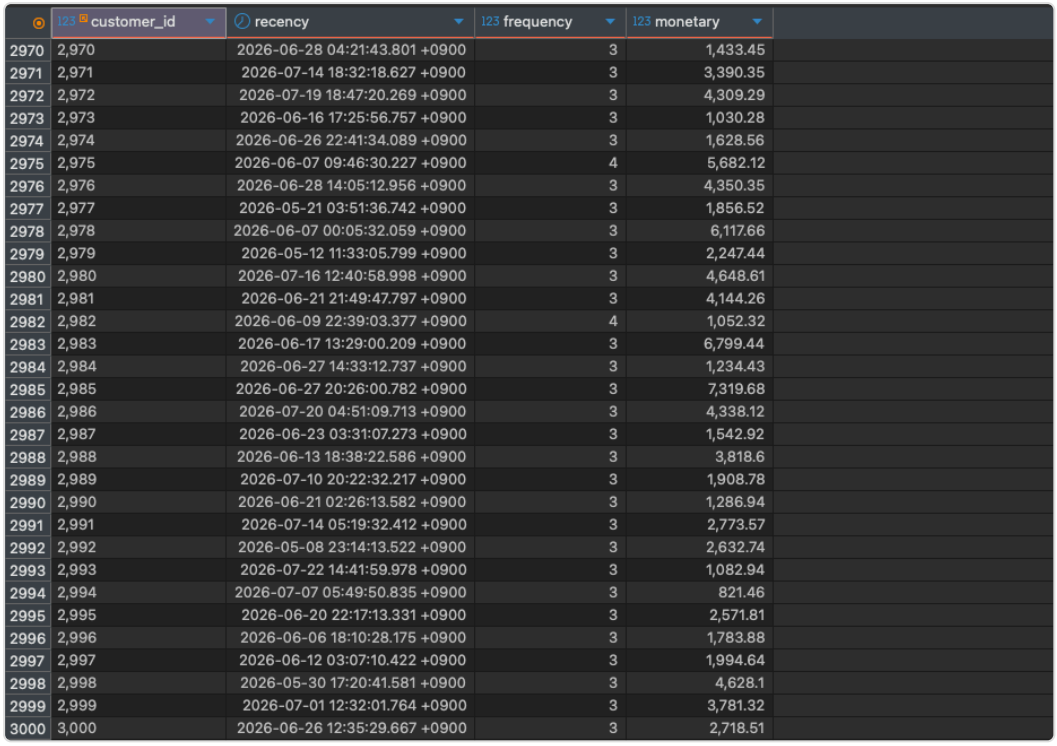
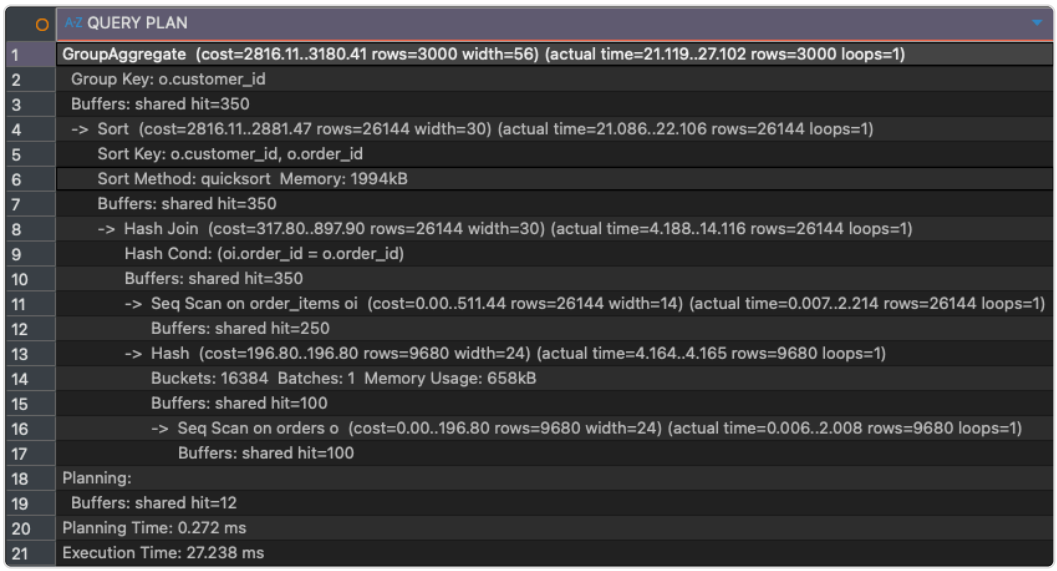
Q5) RFM (최근성 / 빈도 / 금액)

쿼리 의도: 고객별로 최근 구매 시점(Recency), 구매 빈도(Frequency), 총 구매 금액(Monetary)을 구한다.

튜닝 전 문제점 `orders`와 `order_items`를 통째로 조인한 뒤 고객별로 집계했다. `order_items`가 `orders`에 대해 1:N 관계이므로 조인 결과가 fan-out되어, 주문 단위 지표인 recency/frequency까지 부풀려진 행 위에서 계산되고, 이를 바로잡기 위해 `COUNT(DISTINCT o.order_id)`로 억지로 중복을 제거해야 했다.

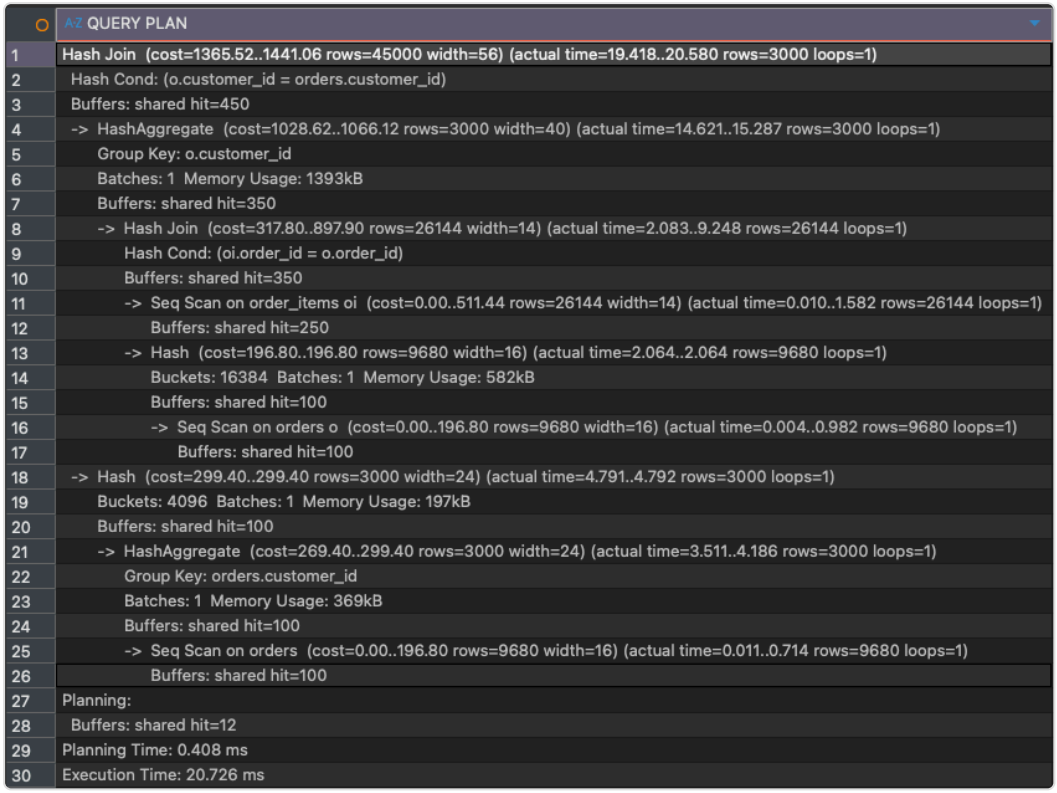
튜닝 내용 - recency/frequency는 `orders` 테이블만으로 먼저 집계(`WITH rf`) - monetary(금액)는 `order_items`와 조인해 별도로 집계(`WITH monetary`) - 마지막에 고객 단위로 두 결과를 병합

효과: fan-out된 결과 위에서 집계하지 않으므로 중간 결과 행 수가 감소하고, `DISTINCT`로 중복을 걷어내는 우회가 필요 없어진다.

구분	스크린샷
결과 확인 (튜닝 전/후 공통)	
튜닝 전 EXPLAIN ANALYZE	

	customer_id	recency	frequency	monetary
2970	2,970	2026-06-28 04:21:43.801 +0900	3	1,433.45
2971	2,971	2026-07-14 18:32:18.627 +0900	3	3,390.35
2972	2,972	2026-07-19 18:47:20.269 +0900	3	4,309.29
2973	2,973	2026-06-16 17:25:56.757 +0900	3	1,030.28
2974	2,974	2026-06-26 22:41:34.089 +0900	3	1,628.56
2975	2,975	2026-06-07 09:46:30.227 +0900	4	5,682.12
2976	2,976	2026-06-28 14:05:12.956 +0900	3	4,350.35
2977	2,977	2026-05-21 03:51:36.742 +0900	3	1,856.52
2978	2,978	2026-06-07 00:05:32.059 +0900	3	6,117.66
2979	2,979	2026-05-12 11:33:05.799 +0900	3	2,247.44
2980	2,980	2026-07-16 12:40:58.998 +0900	3	4,648.61
2981	2,981	2026-06-21 21:49:47.797 +0900	3	4,144.26
2982	2,982	2026-06-09 22:39:03.377 +0900	4	1,052.32
2983	2,983	2026-06-17 13:29:00.209 +0900	3	6,799.44
2984	2,984	2026-06-27 14:33:12.737 +0900	3	1,234.43
2985	2,985	2026-06-27 20:26:00.782 +0900	3	7,319.68
2986	2,986	2026-07-20 04:51:09.713 +0900	3	4,338.12
2987	2,987	2026-06-23 03:31:07.273 +0900	3	1,542.92
2988	2,988	2026-06-13 18:38:22.586 +0900	3	3,818.6
2989	2,989	2026-07-10 20:22:32.217 +0900	3	1,908.78
2990	2,990	2026-06-21 02:26:13.582 +0900	3	1,286.94
2991	2,991	2026-07-14 05:19:32.412 +0900	3	2,773.57
2992	2,992	2026-05-08 23:14:13.522 +0900	3	2,632.74
2993	2,993	2026-07-22 14:41:59.978 +0900	3	1,082.94
2994	2,994	2026-07-07 05:49:50.835 +0900	3	821.46
2995	2,995	2026-06-20 22:17:13.331 +0900	3	2,571.81
2996	2,996	2026-06-06 18:10:28.175 +0900	3	1,783.88
2997	2,997	2026-06-12 03:07:10.422 +0900	3	1,994.64
2998	2,998	2026-05-30 17:20:41.581 +0900	3	4,628.1
2999	2,999	2026-07-01 12:32:01.764 +0900	3	3,781.32
3000	3,000	2026-06-26 12:35:29.667 +0900	3	2,718.51

	QUERY PLAN
1	GroupAggregate (cost=2816.11..3180.41 rows=3000 width=56) (actual time=21.119..27.102 rows=3000 loops=1)
2	Group Key: o.customer_id
3	Buffers: shared hit=350
4	-> Sort (cost=2816.11..2881.47 rows=26144 width=30) (actual time=21.086..22.106 rows=26144 loops=1)
5	Sort Key: o.customer_id, o.order_id
6	Sort Method: quicksort Memory: 1994kB
7	Buffers: shared hit=350
8	-> Hash Join (cost=317.80..897.90 rows=26144 width=30) (actual time=4.188..14.116 rows=26144 loops=1)
9	Hash Cond: (oi.order_id = o.order_id)
10	Buffers: shared hit=350
11	-> Seq Scan on order_items oi (cost=0.00..511.44 rows=26144 width=14) (actual time=0.007..2.214 rows=26144 loops=1)
12	Buffers: shared hit=250
13	-> Hash (cost=196.80..196.80 rows=9680 width=24) (actual time=4.164..4.165 rows=9680 loops=1)
14	Buckets: 16384 Batches: 1 Memory Usage: 658kB
15	Buffers: shared hit=100
16	-> Seq Scan on orders o (cost=0.00..196.80 rows=9680 width=24) (actual time=0.006..2.008 rows=9680 loops=1)
17	Buffers: shared hit=100
18	Planning:
19	Buffers: shared hit=12
20	Planning Time: 0.272 ms
21	Execution Time: 27.238 ms

구분	스크린샷
튜닝 후 EXPLAIN ANALYZE	

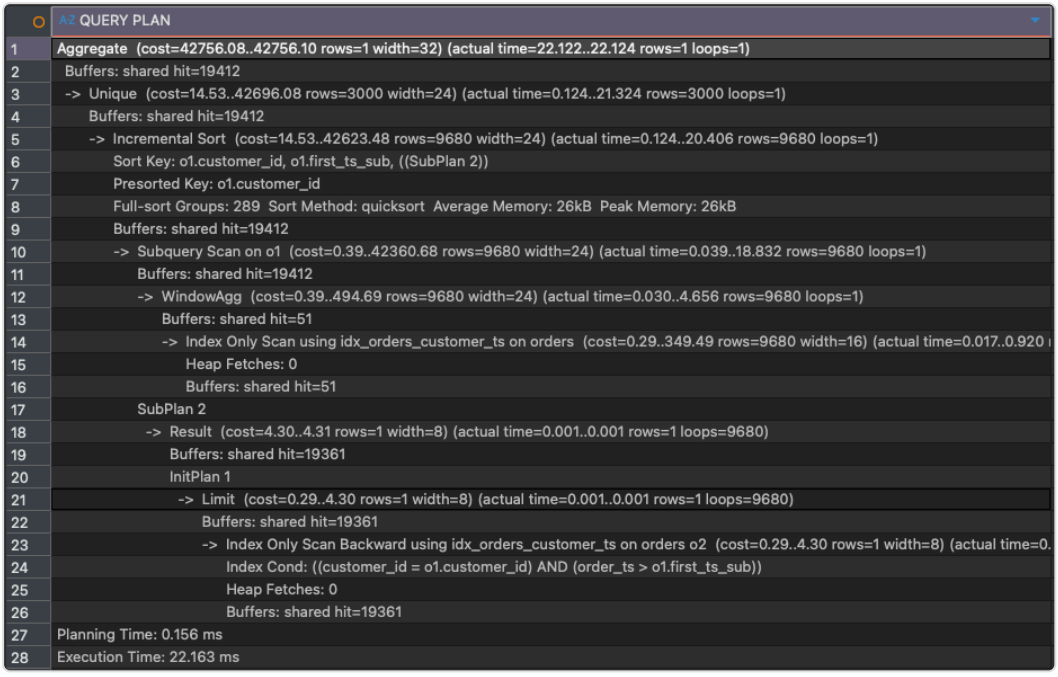
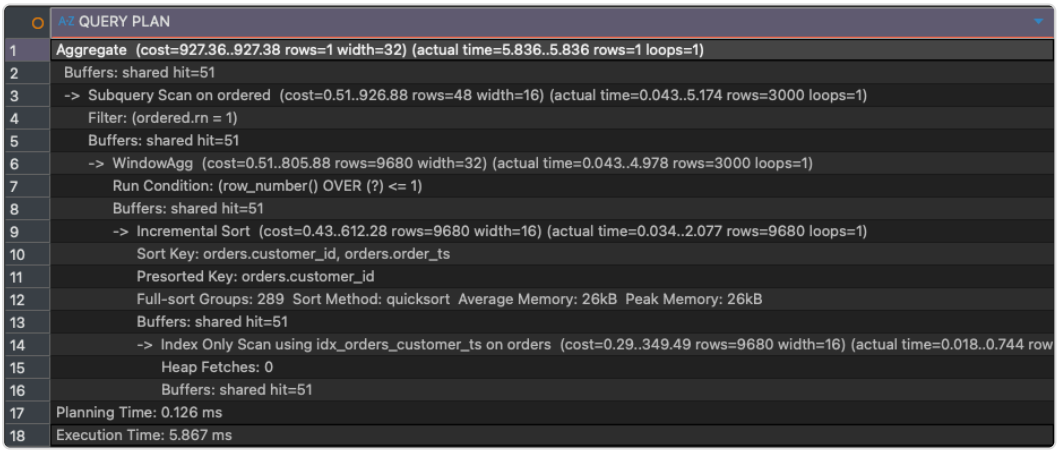
Q6) 첫 구매 후 30일 내 재구매율

쿼리 의도: 고객의 첫 주문 시각을 기준으로, 30일 이내에 두 번째 주문이 발생한 비율을 계산.

튜닝 전 문제점 고객마다 "첫 주문 이후 최소 주문 시각(다음 주문)"을 구하는 상관 서브쿼리를 반복 실행했다. 고객 수만큼 `orders` 를 반복 스캔하는 N+1 패턴이다.

튜닝 내용 - `LEAD(order_ts) OVER (PARTITION BY customer_id ORDER BY order_ts)` 윈도우 함수로 고객별 다음 주문 시각을 한 번의 스캔으로 계산 - `ROW_NUMBER() = 1` (첫 주문)인 행만 남겨 재구매 여부 판정

효과: 반복되는 상관 서브쿼리 스캔이 제거되고, `(customer_id, order_ts)` 인덱스가 있으면 정렬된 스캔 한 번으로 처리 가능해진다.

구분	스크린샷
결과 확인 (튜닝 전/후 공통)	
튜닝 전 EXPLAIN ANALYZE	
튜닝 후 EXPLAIN ANALYZE	

Q7) 재고 임계치 미달 상품

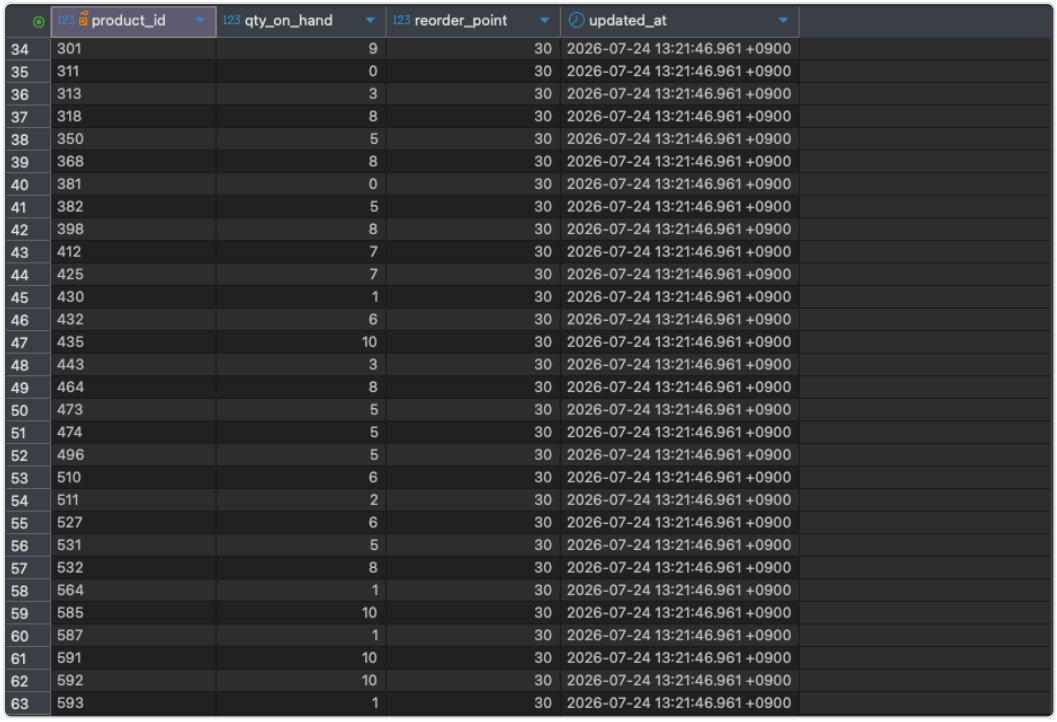
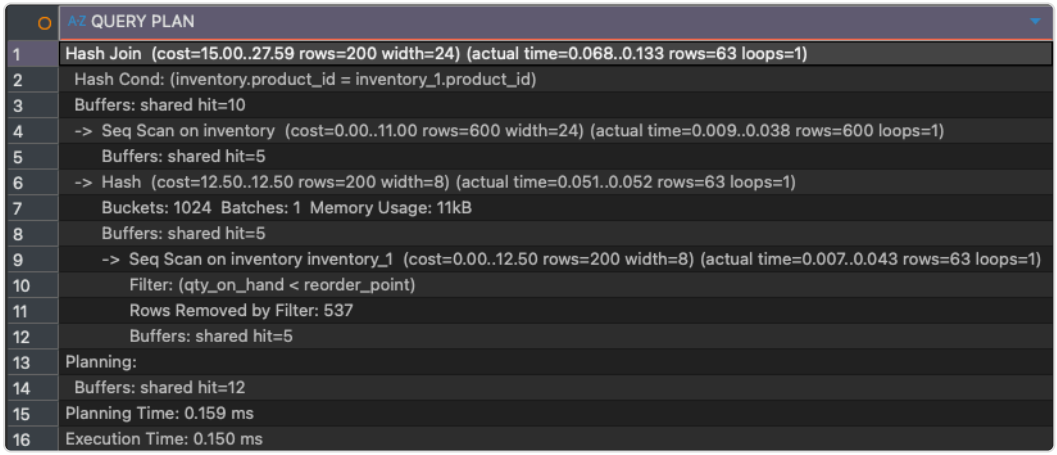
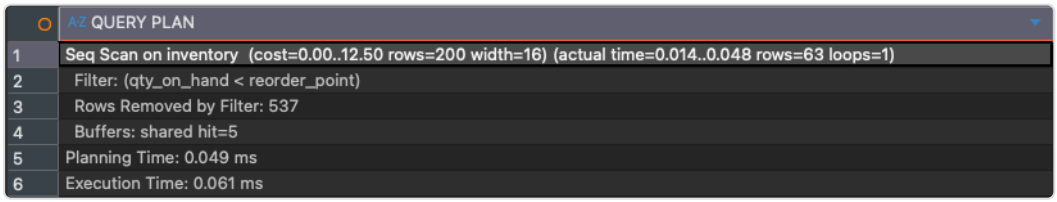
쿼리 의도: 현재 재고 수량(`qty_on_hand`)이 재주문 임계치(`reorder_point`)보다 낮은, 곧 품절될 위험이 있는 상품을 조회.

튜닝 전 문제점 `WHERE product_id IN (SELECT product_id FROM inventory WHERE qty_on_hand < reorder_point)` 처럼 같은 테이블(`inventory`)을 자기 자신에 대한 서브쿼리로 감싸, 불필요하게 두 번 스캔했다.

튜닝 내용 - 서브쿼리를 제거하고 `WHERE qty_on_hand < reorder_point` 조건을 직접 적용

효과: 테이블 스캔이 1회로 감소한다.

참고: `inventory`가 약 600행 정도의 소규모 테이블이라, 인덱스를 추가해도 옵티마이저가 여전히 Seq Scan을 선택할 수 있다. 이는 소규모 테이블에서 인덱스 스캔보다 Seq Scan이 더 저렴하게 판단된 정상적인 선택이다.

구분	스크린샷
결과 확인 (튜닝 전/후 공통)	
튜닝 전 EXPLAIN ANALYZE	
튜닝 후 EXPLAIN ANALYZE	

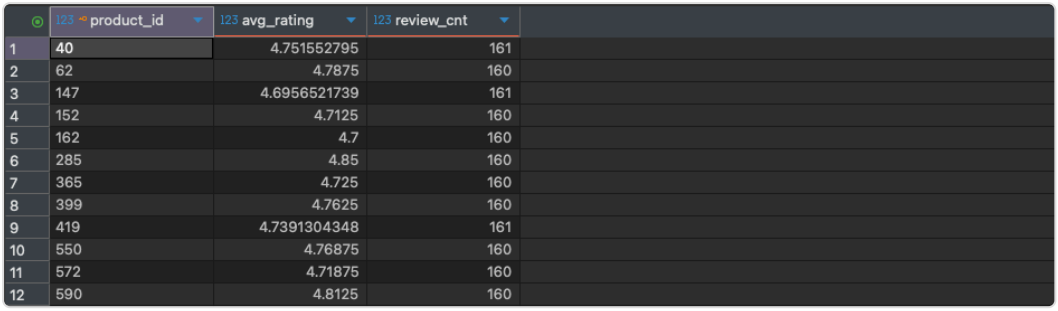
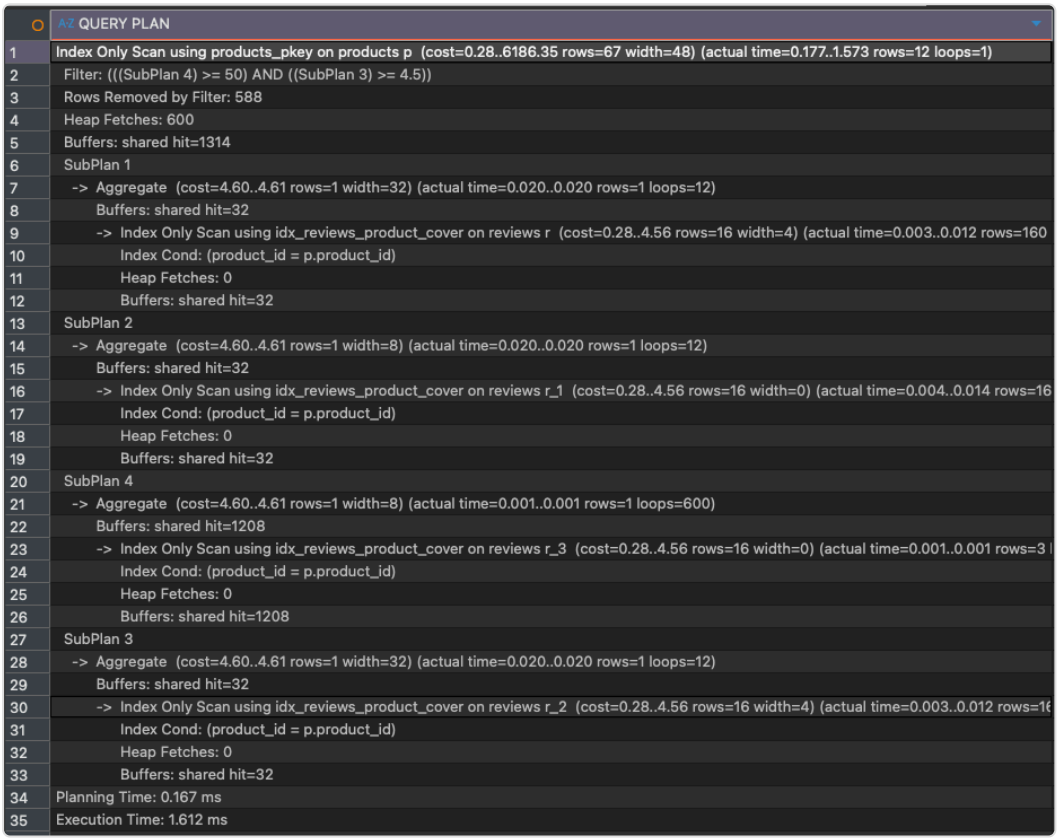
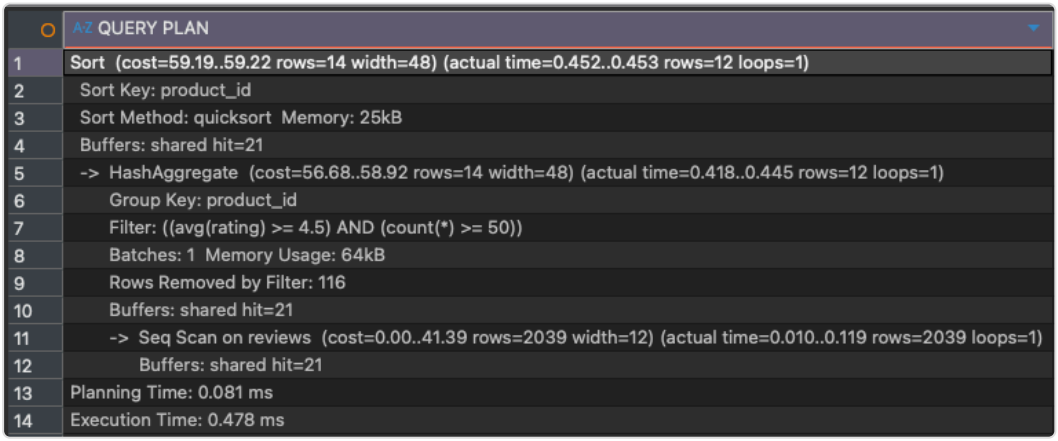
Q8) 리뷰 4.5 이상 & 50개 이상 효자상품

쿼리 의도: 평균 평점 4.5 이상이면서 리뷰 수 50개 이상인, 반응이 좋고 리뷰도 많은 "효자상품"을 조회.

튜닝 전 문제점 상품마다 `AVG(rating)`, `COUNT(*)` 상관 서브쿼리를 `SELECT` 절과 `WHERE` 절에서 각각 반복 호출했다. 상품 수만큼 `reviews` 테이블을 반복 스캔하는 N+1 패턴이다.

튜닝 내용 - `reviews`를 `GROUP BY product_id`로 한 번에 집계하고 `HAVING AVG(rating) >= 4.5 AND COUNT(*) >= 50`으로 조건 필터 - `CREATE INDEX idx_reviews_product_cover ON reviews (product_id) INCLUDE (rating)` 커버링 인덱스 추가

효과: `reviews` 스캔이 1회로 감소하고, 커버링 인덱스 덕분에 힙 접근 없이 인덱스만으로 집계(Index Only Scan)할 수 있다.

구분	스크린샷
결과 확인 (튜닝 전/후 공통)	
튜닝 전 EXPLAIN ANALYZE	
튜닝 후 EXPLAIN ANALYZE	

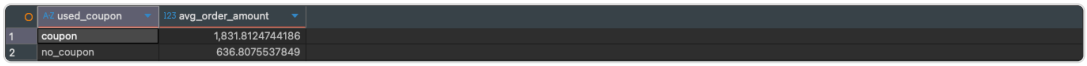
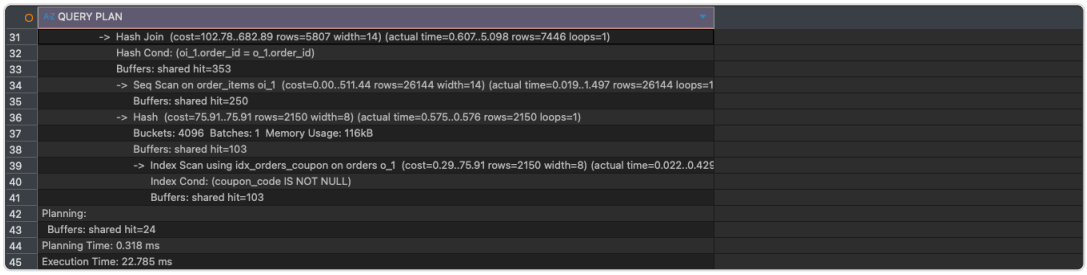
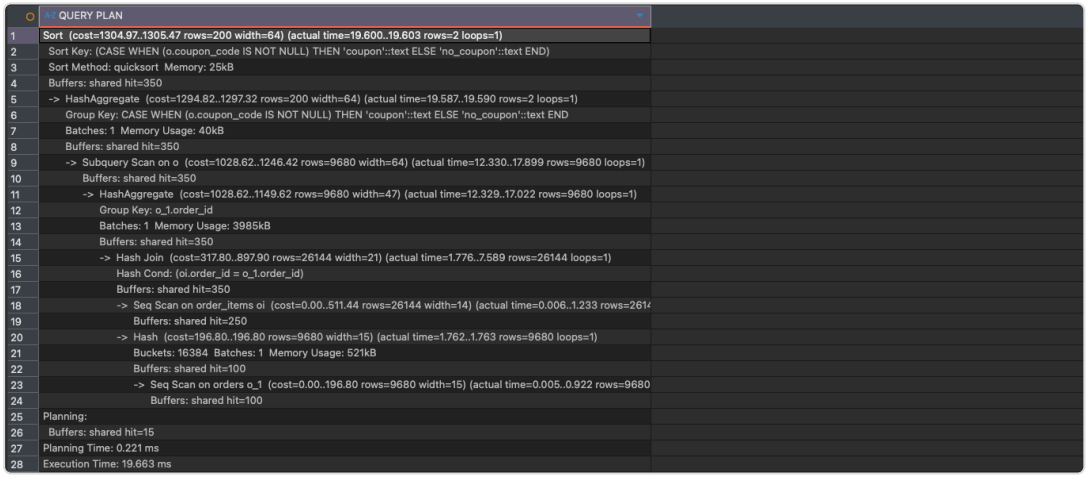
Q9) 쿠폰 사용 영향 (쿠폰 사용 주문 vs 미사용 주문 평균금액)

쿼리 의도: 쿠폰을 사용한 주문과 사용하지 않은 주문의 평균 주문 금액을 비교.

튜닝 전 문제점 쿠폰 있음/없음 두 그룹을 각각 별도 쿼리로 작성한 뒤 `UNION ALL` 로 합쳤다. 이 때문에 `orders` / `order_items` 조인을 두 번 반복 수행하게 된다.

튜닝 내용 - `CASE WHEN o.coupon_code IS NOT NULL THEN 'coupon' ELSE 'no_coupon' END` 으로 그룹 라벨을 만들고, 한 번의 `GROUP BY` 에서 두 그룹을 동시에 집계 - `CREATE INDEX idx_orders_coupon ON orders (coupon_code)` 추가

효과: 테이블 스캔이 1회로 줄어 두 그룹을 동시에 계산한다.

구분	스크린샷
결과 확인 (튜닝 전/후 공통)	
튜닝 전 EXPLAIN ANALYZE	
튜닝 후 EXPLAIN ANALYZE	

Q10) 상위 1% 고객의 최근 60일 매출

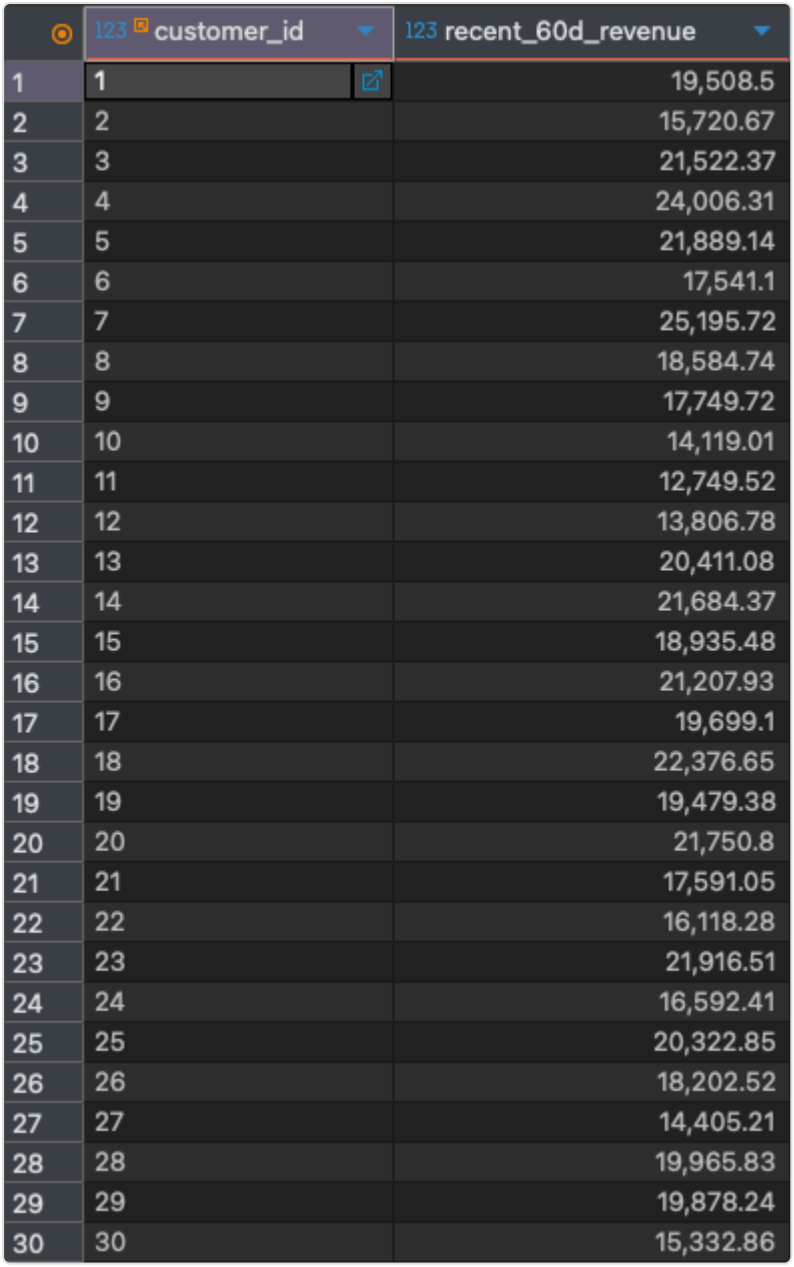
쿼리 의도: 누적 매출 기준 상위 1% 고객을 가려내고, 이들의 최근 60일 매출을 조회.

튜닝 전 문제점 전체 고객 매출 계산, 백분위(`PERCENTILE_CONT(0.99)`) 계산, 최근 60일 매출 계산이 한 쿼리 안에 뒤섞여 있어, 최종적으로는 상위 1%에 들지 못하는 고객들까지 전부 조인·집계한 뒤에야 필터링되는 비효율이 있었다.

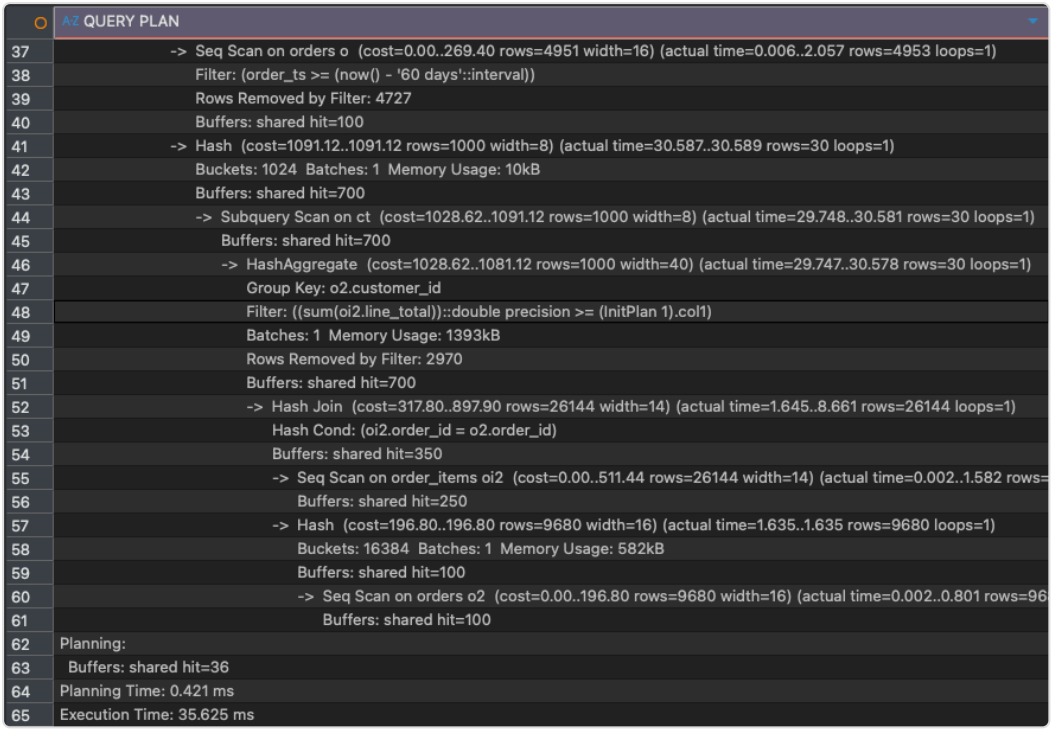
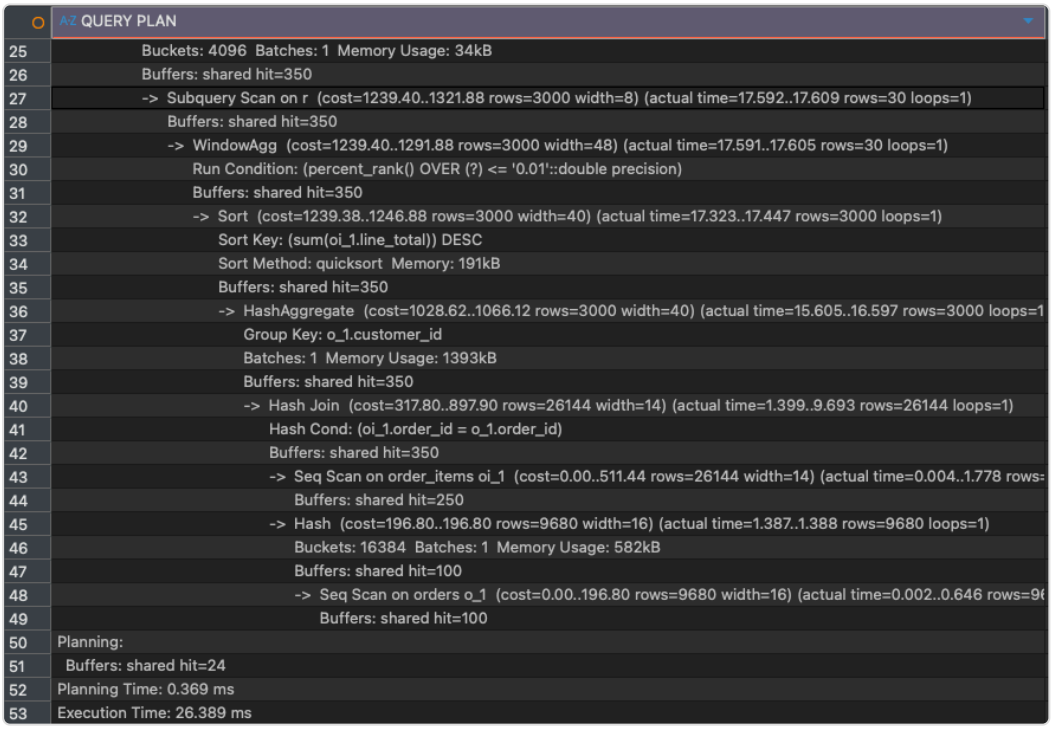
튜닝 내용 - `PERCENT_RANK() OVER (ORDER BY SUM(line_total) DESC)` 로 상위 1% 고객 집합 (`top_customers`)을 CTE로 먼저 확정 - 확정된 소수의 고객에 대해서만 최근 60일 매출을 다시 조회

효과: 최종적으로 필요한 소수 고객 집합만 재스캔하면 되므로, 처리해야 하는 데이터량이 대폭 감소한다.

검증: `customer_id` 가 대부분 1~30 사이로 나와야 정상 (seed 데이터의 heavy customer들).

구분	스크린샷
결과 확인 (튜닝 전/후 공통)	

	customer_id	recent_60d_revenue
1	1	19,508.5
2	2	15,720.67
3	3	21,522.37
4	4	24,006.31
5	5	21,889.14
6	6	17,541.1
7	7	25,195.72
8	8	18,584.74
9	9	17,749.72
10	10	14,119.01
11	11	12,749.52
12	12	13,806.78
13	13	20,411.08
14	14	21,684.37
15	15	18,935.48
16	16	21,207.93
17	17	19,699.1
18	18	22,376.65
19	19	19,479.38
20	20	21,750.8
21	21	17,591.05
22	22	16,118.28
23	23	21,916.51
24	24	16,592.41
25	25	20,322.85
26	26	18,202.52
27	27	14,405.21
28	28	19,965.83
29	29	19,878.24
30	30	15,332.86

구분	스크린샷
튜닝 전 EXPLAIN ANALYZE	
튜닝 후 EXPLAIN ANALYZE	

2. 3가지 조인에 대한 차이점 정리: Hash Join / Nested Loop / Bitmap Heap Scan

먼저 짚어야 할 점: **Hash Join과 Nested Loop Join은 "두 릴레이션을 어떻게 결합할지"를 결정하는 조인 알고리즘**이고, **Bitmap Heap Scan은 "한 테이블에서 행을 어떻게 읽어올지"를 결정하는 스캔(접근) 방식**이다. 성격이 다른 개념이지만,

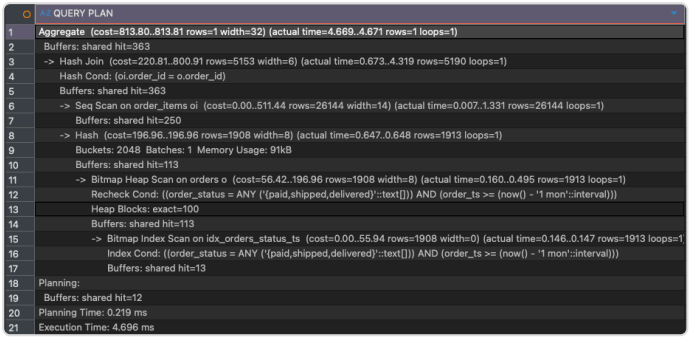
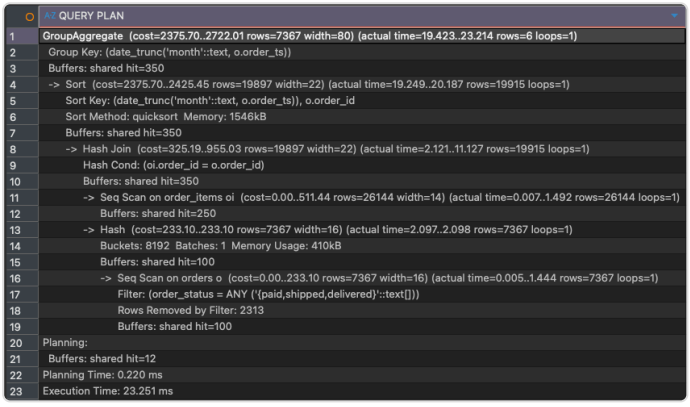
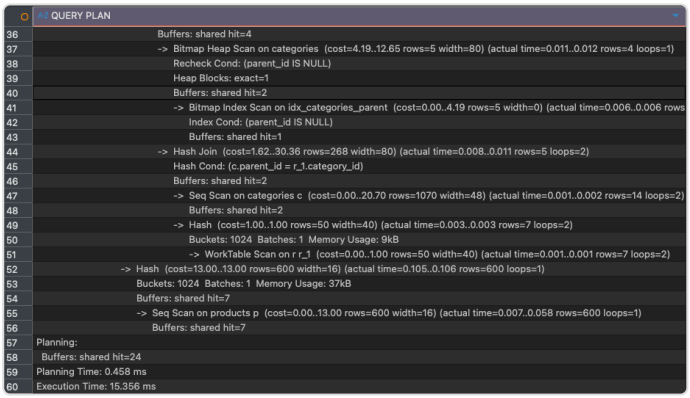
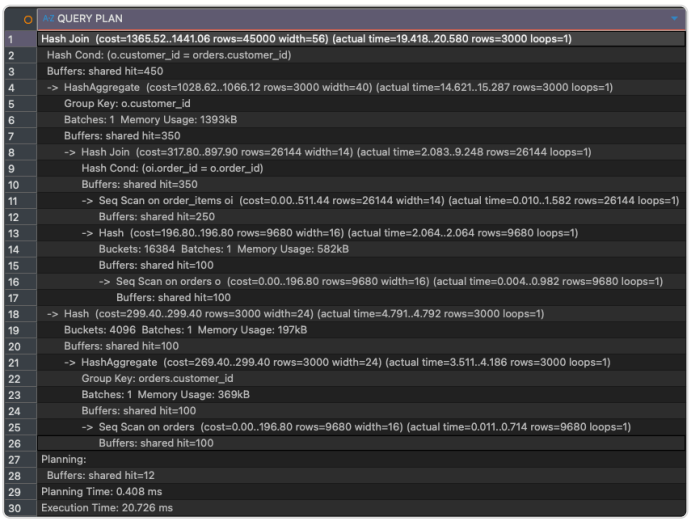
Bitmap Heap Scan은 실제로는 Hash Join·Nested Loop의 입력 쪽에서 자주 함께 등장하기 때문에 실행계획을 읽을 때는 세트로 이해하는 것이 실용적이다.

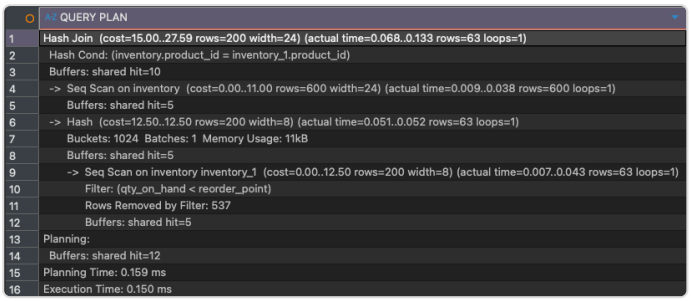
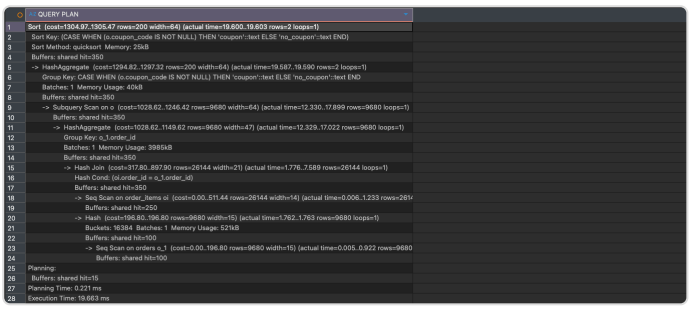
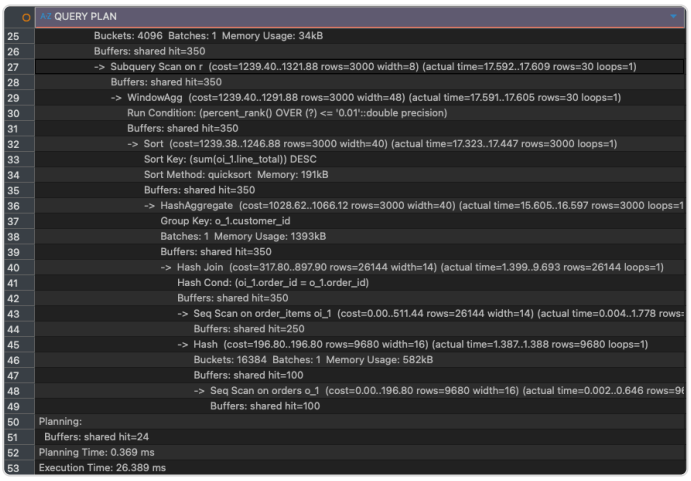
2-1. 개념 정리

구분	동작 방식	유리한 상황	불리한 상황
Hash Join	두 릴레이션 중 작은 쪽으로 해시 테이블을 메모리에 만든 뒤(Hash), 큰 쪽을 한 번 스캔하며 해시 테이블에 프로브(probe)	동등 조인(equi-join)이고, 조인 키에 쓸만한 인덱스가 없거나 양쪽 다 대용량일 때	`work_mem`이 부족해 해시 테이블이 여러 배치(Batch)로 나뉘어 디스크를 쓸 때, 비동등 조인일 때
Nested Loop Join	외부(outer) 릴레이션의 각 행마다 내부(inner) 릴레이션을 반복 스캔(또는 인덱스로 탐색)	outer 쪽 행 수가 적고(필터로 크게 좁혀졌고), inner 쪽 조인 키에 인덱스가 있을 때	outer/inner 모두 행 수가 많고 inner에 인덱스가 없을 때 → 반복 횟수만큼 비용이 곱해짐
Bitmap Heap Scan	인덱스에서 조건에 맞는 힙 페이지 위치들을 먼저 비트맵으로 수집(Bitmap Index Scan) 한 뒤, 힙 페이지를 물리적 순서로 방문해 실제 행을 가져옴(필요시 Recheck)	조건에 맞는 행의 비율이 "적음"과 "대부분"의 중간 정도라 Index Scan(랜덤 I/O 과다)도 Seq Scan(전체 스캔)도 최선이 아닐 때	조건에 맞는 행이 거의 없으면 Index Scan이, 대부분이면 Seq Scan이 더 나음

2-2. 본 실습에서 실제로 관측된 사례

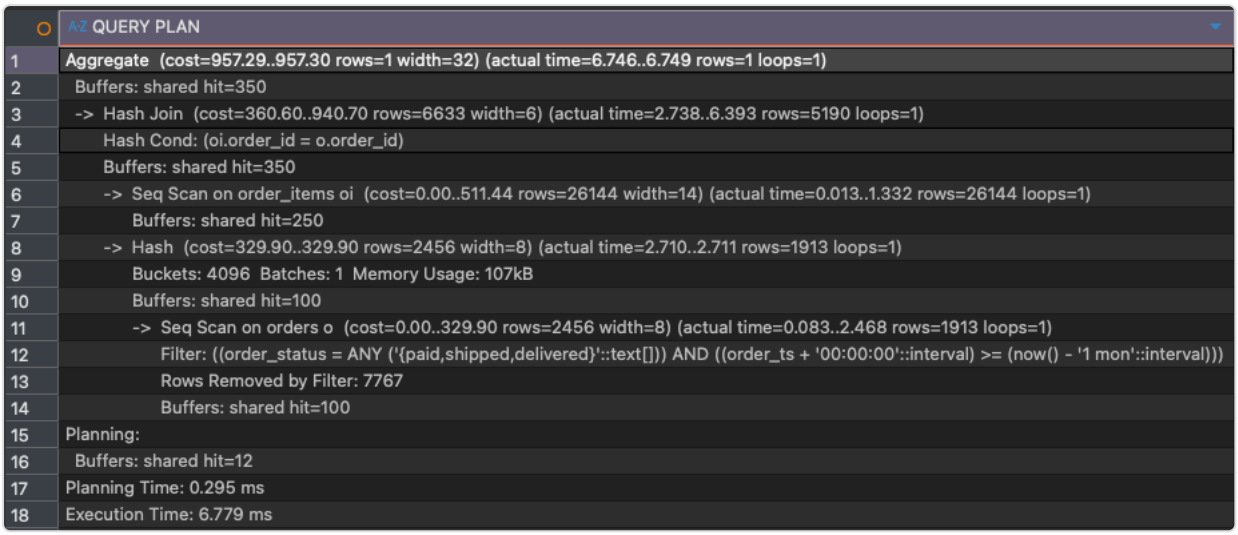
Hash Join — `orders`와 `order_items`처럼 조인 키(`order_id`)에 전용 인덱스가 없고 양쪽 다 수천\~수만 행인 동등 조인에서는 튜닝 전/후 가릴 것 없이 항상 Hash Join이 선택되었다.

문항	Hash Cond	근거
Q1	`oi.order_id = o.order_id`	
Q2	`oi.order_id = o.order_id` (튜닝 전)	
Q3	`oi.order_id = o.order_id`, `c.parent_id = r_1.category_id`	
Q5	`o.customer_id = orders.customer_id` (rf ⋈ monetary)	

문항	Hash Cond	근거
Q7	`inventory.product_id = inventory_1.product_id` (튜닝 전, 자기 자신과 조인)	
Q9	`oi.order_id = o_1.order_id`	
Q10	`oi.order_id = o.order_id`	

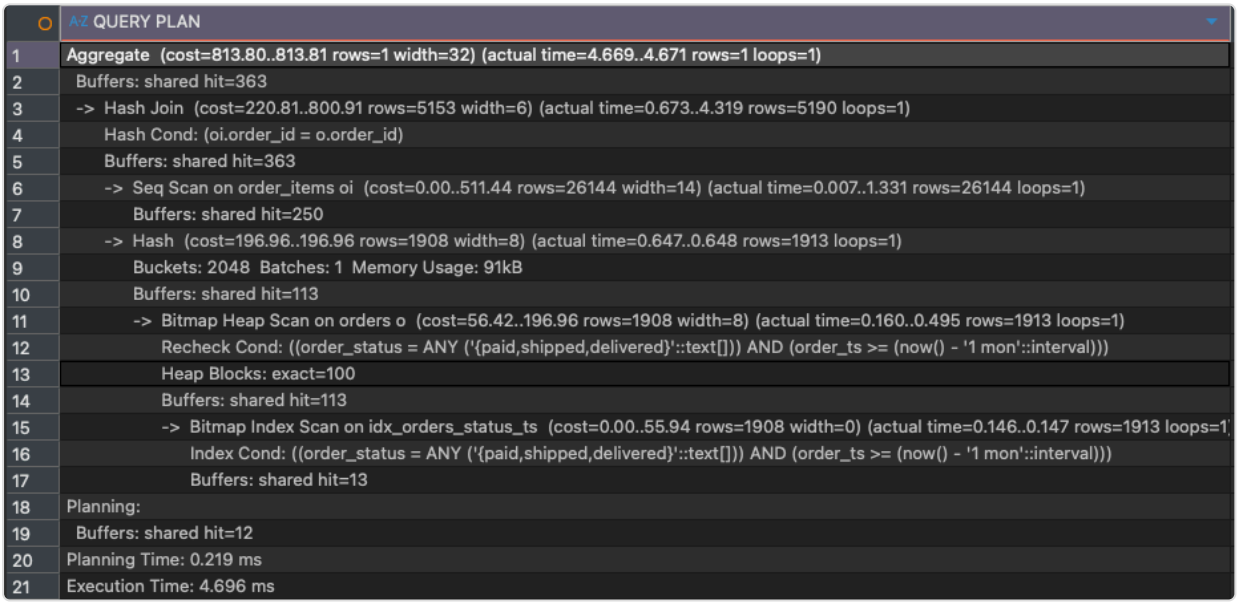
Bitmap Heap Scan — 단일 테이블에서 조건에 맞는 행의 비율이 중간 정도일 때 등장했다. 대표적으로 Q1 튜닝 전/후 비교에서 그 역할이 뚜렷하게 드러난다.

- Q1 튜닝 전: `Seq Scan on orders` + `Filter`로 2,456행을 다 읽고서 조건에 안 맞는 것을 걸러냄 (Rows Removed by Filter: 767)
- Q1 튜닝 후: `idx_orders_status_ts` 추가 후 `Bitmap Index Scan on idx_orders_status_ts`로 조건에 맞는 페이지 위치를 먼저 찾고, `Bitmap Heap Scan on orders`로 그 페이지들만 방문 (Heap Blocks: exact=100)
- **중요한 관찰**: 이때도 **상위 조인 방식은 Hash Join 그대로 유지**되었다 — 즉 인덱스 튜닝이 바꾼 것은 "조인 알고리즘"이 아니라 "조인에 넘겨줄 입력 행을 Hash Join에게 더 싸게 공급하는 스캔 방식"이었다. (



	QUERY PLAN
1	Aggregate (cost=957.29..957.30 rows=1 width=32) (actual time=6.746..6.749 rows=1 loops=1)
2	Buffers: shared hit=350
3	-> Hash Join (cost=360.60..940.70 rows=6633 width=6) (actual time=2.738..6.393 rows=5190 loops=1)
4	Hash Cond: (oi.order_id = o.order_id)
5	Buffers: shared hit=350
6	-> Seq Scan on order_items oi (cost=0.00..511.44 rows=26144 width=14) (actual time=0.013..1.332 rows=26144 loops=1)
7	Buffers: shared hit=250
8	-> Hash (cost=329.90..329.90 rows=2456 width=8) (actual time=2.710..2.711 rows=1913 loops=1)
9	Buckets: 4096 Batches: 1 Memory Usage: 107kB
10	Buffers: shared hit=100
11	-> Seq Scan on orders o (cost=0.00..329.90 rows=2456 width=8) (actual time=0.083..2.468 rows=1913 loops=1)
12	Filter: ((order_status = ANY ('{paid,shipped,delivered}'::text[])) AND ((order_ts + '00:00:00'::interval) >= (now() - '1 mon'::interval)))
13	Rows Removed by Filter: 7767
14	Buffers: shared hit=100
15	Planning:
16	Buffers: shared hit=12
17	Planning Time: 0.295 ms
18	Execution Time: 6.779 ms

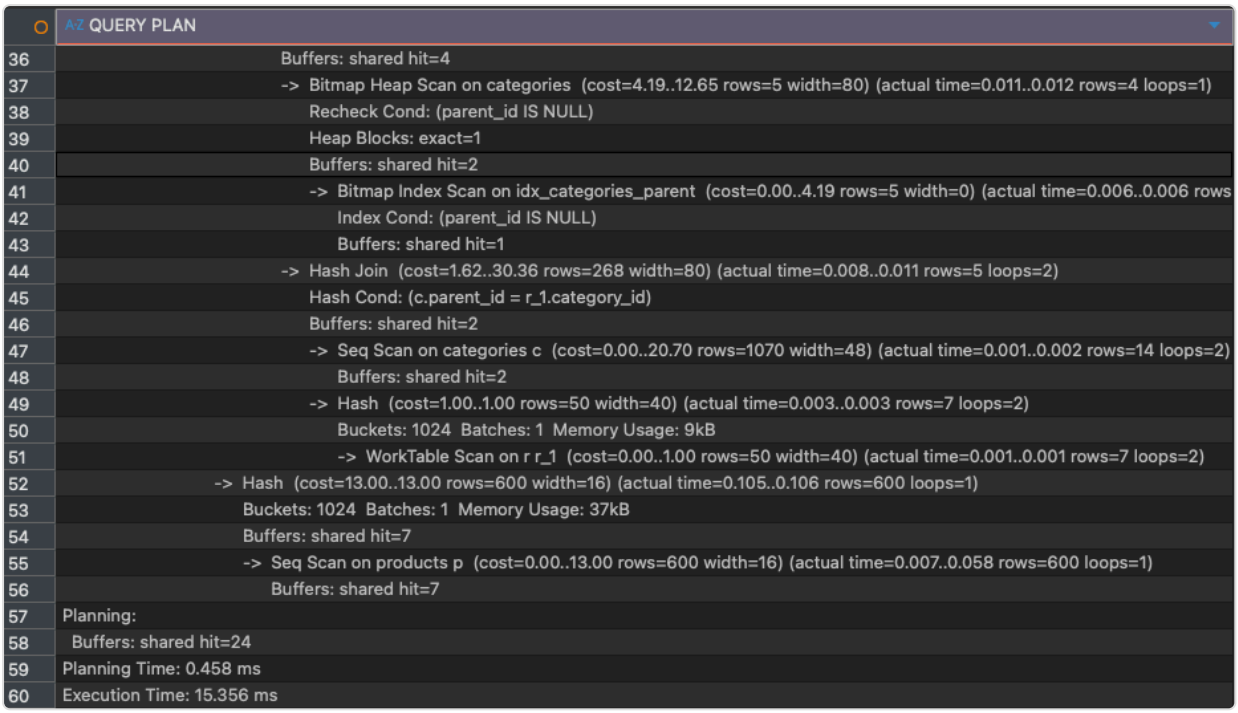
→



	QUERY PLAN
1	Aggregate (cost=813.80..813.81 rows=1 width=32) (actual time=4.669..4.671 rows=1 loops=1)
2	Buffers: shared hit=363
3	-> Hash Join (cost=220.81..800.91 rows=5153 width=6) (actual time=0.673..4.319 rows=5190 loops=1)
4	Hash Cond: (oi.order_id = o.order_id)
5	Buffers: shared hit=363
6	-> Seq Scan on order_items oi (cost=0.00..511.44 rows=26144 width=14) (actual time=0.007..1.331 rows=26144 loops=1)
7	Buffers: shared hit=250
8	-> Hash (cost=196.96..196.96 rows=1908 width=8) (actual time=0.647..0.648 rows=1913 loops=1)
9	Buckets: 2048 Batches: 1 Memory Usage: 91kB
10	Buffers: shared hit=113
11	-> Bitmap Heap Scan on orders o (cost=56.42..196.96 rows=1908 width=8) (actual time=0.160..0.495 rows=1913 loops=1)
12	Recheck Cond: ((order_status = ANY ('{paid,shipped,delivered}'::text[])) AND (order_ts >= (now() - '1 mon'::interval)))
13	Heap Blocks: exact=100
14	Buffers: shared hit=113
15	-> Bitmap Index Scan on idx_orders_status_ts (cost=0.00..55.94 rows=1908 width=0) (actual time=0.146..0.147 rows=1913 loops=1
16	Index Cond: ((order_status = ANY ('{paid,shipped,delivered}'::text[])) AND (order_ts >= (now() - '1 mon'::interval)))
17	Buffers: shared hit=13
18	Planning:
19	Buffers: shared hit=12
20	Planning Time: 0.219 ms
21	Execution Time: 4.696 ms

)

- Q3 튜닝 후에도 카테고리 트리의 루트 노드를 찾는 부분에서 `Bitmap Heap Scan on categories` + `Bitmap Index Scan on idx_categories_parent` 가 동일한 패턴으로 등장 (

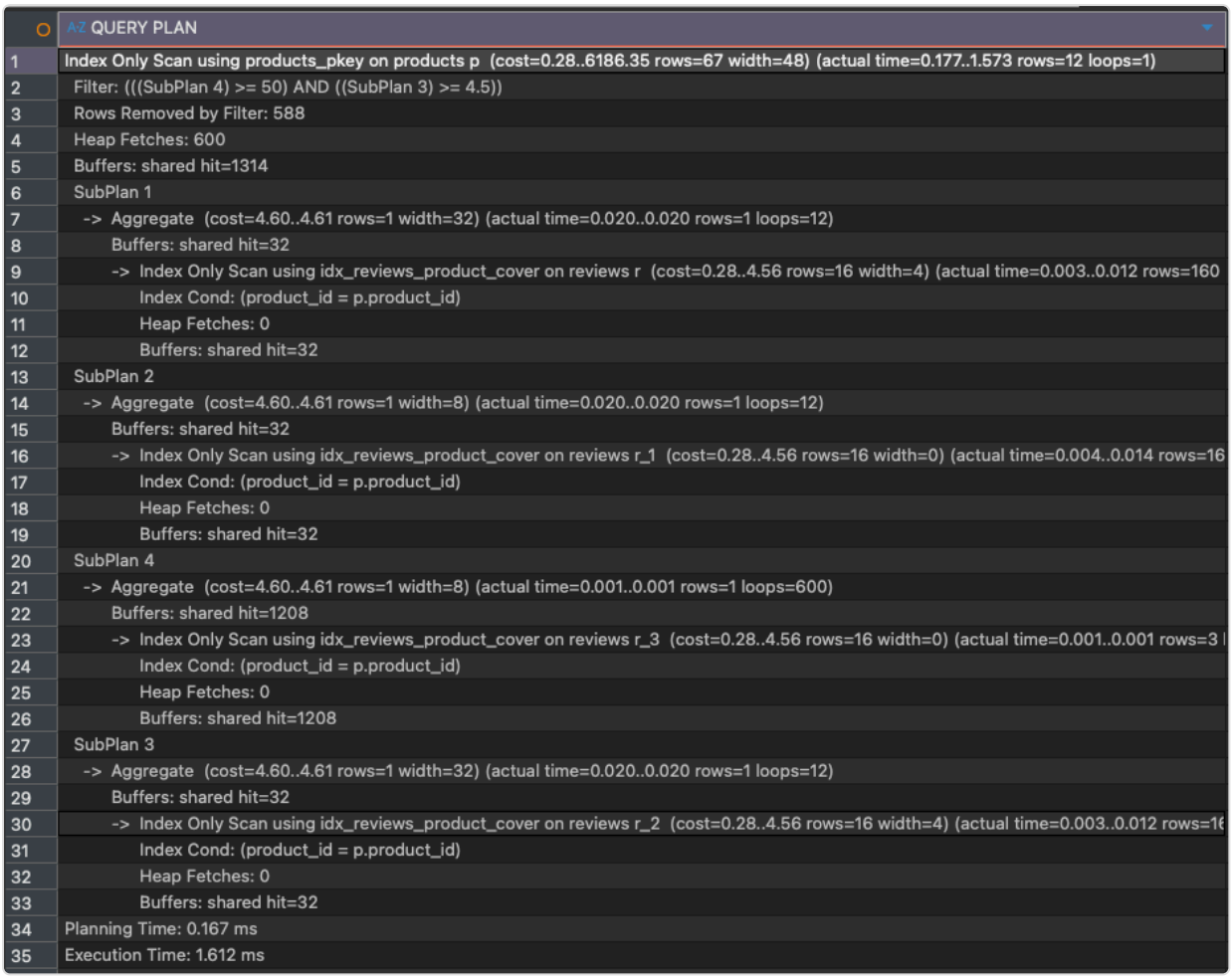


```
QUERY PLAN
                                Buffers: shared hit=4
                                -> Bitmap Heap Scan on categories  (cost=4.19..12.65 rows=5 width=80) (actual time=0.011..0.012 rows=4 loops=1)
                                   Recheck Cond: (parent_id IS NULL)
                                   Heap Blocks: exact=1
                                   Buffers: shared hit=2
                                   -> Bitmap Index Scan on idx_categories_parent  (cost=0.00..4.19 rows=5 width=0) (actual time=0.006..0.006 rows
                                      Index Cond: (parent_id IS NULL)
                                      Buffers: shared hit=1
                                -> Hash Join  (cost=1.62..30.36 rows=268 width=80) (actual time=0.008..0.011 rows=5 loops=2)
                                   Hash Cond: (c.parent_id = r_1.category_id)
                                   Buffers: shared hit=2
                                   -> Seq Scan on categories c  (cost=0.00..20.70 rows=1070 width=48) (actual time=0.001..0.002 rows=14 loops=2)
                                      Buffers: shared hit=2
                                   -> Hash  (cost=1.00..1.00 rows=50 width=40) (actual time=0.003..0.003 rows=7 loops=2)
                                      Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                      -> WorkTable Scan on r r_1  (cost=0.00..1.00 rows=50 width=40) (actual time=0.001..0.001 rows=7 loops=2)
                          -> Hash  (cost=13.00..13.00 rows=600 width=16) (actual time=0.105..0.106 rows=600 loops=1)
                             Buckets: 1024  Batches: 1  Memory Usage: 37kB
                             Buffers: shared hit=7
                             -> Seq Scan on products p  (cost=0.00..13.00 rows=600 width=16) (actual time=0.007..0.058 rows=600 loops=1)
                                Buffers: shared hit=7
Planning:
 Buffers: shared hit=24
Planning Time: 0.458 ms
Execution Time: 15.356 ms
```

)

Nested Loop Join — 이번 실습에서 캡처한 20개의 실행계획(Q1~Q10 튜닝 전/후) 전체를 다시 확인한 결과, **명시적인 Nested Loop 노드는 한 건도 관측되지 않았다.**

- **직접적인 이유(쿼리 구조)**: Nested Loop이 나오려면 애초에 outer 쪽 결과가 소수로 좁혀진 뒤 inner 테이블과 PK/인덱스로 다시 조인하는 지점이 쿼리에 있어야 한다. 그런데 Q1~Q10의 조인은 전부 `orders ⋈ order_items`(또는 `products`, `categories`)처럼 양쪽 다 수천\~수만 행인 동등 조인이고, 상위 1% 고객처럼 결과가 극소수로 좁혀지는 Q10조차 `customers` 테이블을 다시 조인하지 않고 `customer_id`와 매출만 반환하도록 작성했다. 즉 "소수 결과 → PK로 재조회"하는 지점 자체가 우리 쿼리들에는 없었다.
- **간접적인 이유(옵티마이저 판단)**: 설사 그런 조인이 있었더라도, `order_items.order_id`에는 조인 전용 단일 인덱스(`idx_order_items_order`)만 있고 outer 쪽 행 수 자체가 충분히 작아지지 않는 한 옵티마이저는 Hash Join을 항상 더 저렴하다고 판단한다.
- Q8처럼 상품마다 리뷰 통계를 반복 조회하는 상관 서브쿼리(before)는 언뜻 "행마다 반복"이라는 점에서 Nested Loop과 비슷해 보이지만, 실제 플랜에서는 `SubPlan` / `InitPlan` 형태로 표시되며 명시적 `Nested Loop` 조인 노드는 아니었다 (



	QUERY PLAN
1	Index Only Scan using products_pkey on products p (cost=0.28..6186.35 rows=67 width=48) (actual time=0.177..1.573 rows=12 loops=1)
2	Filter: (((SubPlan 4) >= 50) AND ((SubPlan 3) >= 4.5))
3	Rows Removed by Filter: 588
4	Heap Fetches: 600
5	Buffers: shared hit=1314
6	SubPlan 1
7	-> Aggregate (cost=4.60..4.61 rows=1 width=32) (actual time=0.020..0.020 rows=1 loops=12)
8	Buffers: shared hit=32
9	-> Index Only Scan using idx_reviews_product_cover on reviews r (cost=0.28..4.56 rows=16 width=4) (actual time=0.003..0.012 rows=160
10	Index Cond: (product_id = p.product_id)
11	Heap Fetches: 0
12	Buffers: shared hit=32
13	SubPlan 2
14	-> Aggregate (cost=4.60..4.61 rows=1 width=8) (actual time=0.020..0.020 rows=1 loops=12)
15	Buffers: shared hit=32
16	-> Index Only Scan using idx_reviews_product_cover on reviews r_1 (cost=0.28..4.56 rows=16 width=0) (actual time=0.004..0.014 rows=16
17	Index Cond: (product_id = p.product_id)
18	Heap Fetches: 0
19	Buffers: shared hit=32
20	SubPlan 4
21	-> Aggregate (cost=4.60..4.61 rows=1 width=8) (actual time=0.001..0.001 rows=1 loops=600)
22	Buffers: shared hit=1208
23	-> Index Only Scan using idx_reviews_product_cover on reviews r_3 (cost=0.28..4.56 rows=16 width=0) (actual time=0.001..0.001 rows=3
24	Index Cond: (product_id = p.product_id)
25	Heap Fetches: 0
26	Buffers: shared hit=1208
27	SubPlan 3
28	-> Aggregate (cost=4.60..4.61 rows=1 width=32) (actual time=0.020..0.020 rows=1 loops=12)
29	Buffers: shared hit=32
30	-> Index Only Scan using idx_reviews_product_cover on reviews r_2 (cost=0.28..4.56 rows=16 width=4) (actual time=0.003..0.012 rows=16
31	Index Cond: (product_id = p.product_id)
32	Heap Fetches: 0
33	Buffers: shared hit=32
34	Planning Time: 0.167 ms
35	Execution Time: 1.612 ms

).

- **언제 나타나는가**: Q10을 "상위 1% 고객의 customer_id + 이름/이메일까지" 조회하도록 바꿔서 `top_customers`(소수 행)를 `customers` 테이블과 PK로 다시 JOIN하면, outer가 이미 극소수로 좁혀져 있고 inner(`customers`)에 PK 인덱스가 있으므로 Nested Loop Join이 선택될 조건이 된다. 즉 Nested Loop을 관측하지 못한 것은 "데이터가 부족해서"가 아니라 "우리 쿼리가 그런 조인을 요구하지 않아서"이다.

2-3. 요약

	Hash Join	Nested Loop	Bitmap Heap Scan
분류	조인 알고리즘	조인 알고리즘	(단일 테이블) 스캔 방식
본 실습 관측 여부	관측됨 (모든 다중 테이블 조인)	관측 안 됨	관측됨 (인덱스 추가 후 등치+범위 조건 스캔)
관측 안 된 이유	—	쿼리에 "소수 결과 → PK 재조회" 지점이 없었음 (Q10이 customers를 조인하지 않음)	—

---

3. Materialized View 생성 스크립트와 실행 후 결과 화면

3-1. 생성 스크립트

Q2에서 사용한 `mv_daily_gmv` 는 일별 주문수(`order_cnt`)와 매출(`gmv`)을 사전 집계해두는 Materialized View 다. (원본 스키마의 `mv_daily_gmv` 는 `day` / `gmv` 뿐이라 Q2가 요구하는 주문수·AOV까지 커버하도록 재정의했다.)

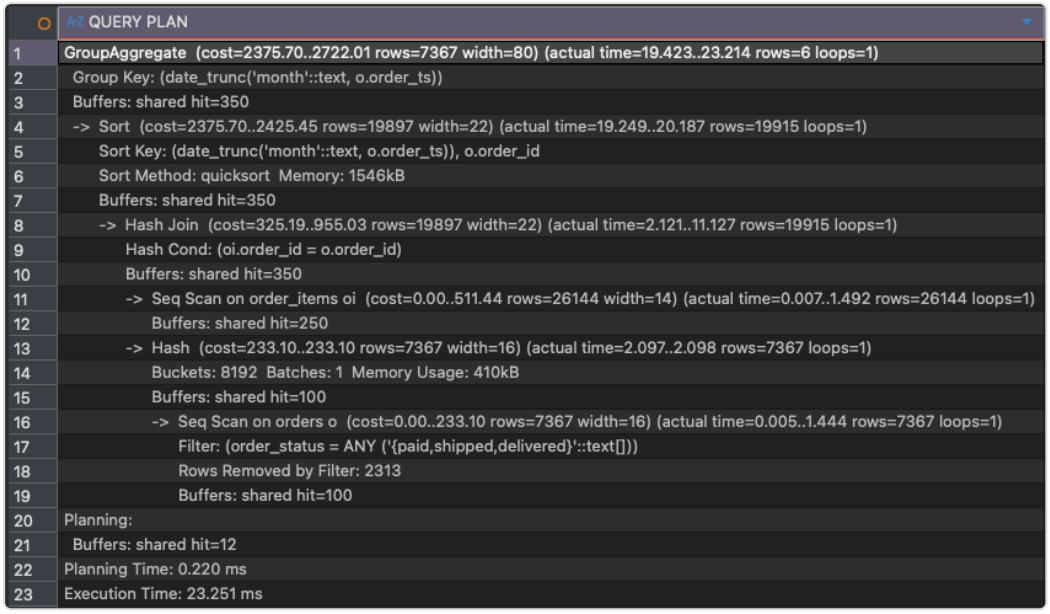
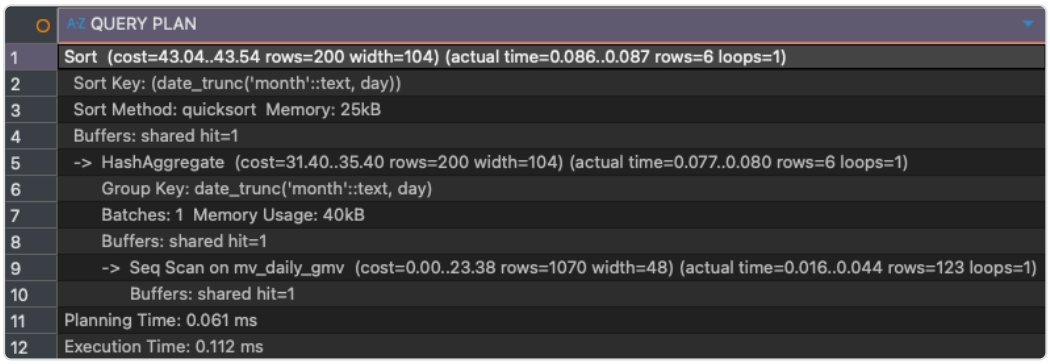
```
DROP MATERIALIZED VIEW IF EXISTS mv_daily_gmv;
CREATE MATERIALIZED VIEW mv_daily_gmv AS
SELECT date_trunc('day', o.order_ts) AS day,
       COUNT(DISTINCT o.order_id) AS order_cnt,
       SUM(oi.line_total) AS gmv
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid','shipped','delivered')
GROUP BY 1;
CREATE INDEX IF NOT EXISTS idx_mv_daily_gmv_day ON mv_daily_gmv (day);
```

3-2. 리포트 질의 가속 효과 (실측)

Q2 EXPLAIN ANALYZE 캡처를 비교하면 가속 효과가 수치로 확인된다.

	실행 계획 요약	실행 시간
튜닝 전 (원본 조인)	`orders` (Seq Scan, Filter로 2,313행 제거) ⋈ `order_items` (Seq Scan, 26,144행) → Hash Join → Sort → GroupAggregate	**23.251 ms**
튜닝 후 (MV 스캔)	`mv_daily_gmv` (Seq Scan, 123행) → HashAggregate → Sort	**0.112 ms**

→ 약 **207배** 실행 시간 단축 (23.251ms → 0.112ms). 원본 조인이 매번 26,144행짜리 `order_items` 를 스캔해야 했던 것에 비해, MV는 이미 하루 단위로 집계된 123행만 스캔하면 되기 때문이다.

구분	스크린샷
튜닝 전 EXPLAIN ANALYZE	
튜닝 후 EXPLAIN ANALYZE	

3-3. 갱신(REFRESH) 전략 검토

Materialized View는 스냅샷이라 원본 데이터(`orders`, `order_items`)가 바뀌어도 자동으로 최신화되지 않는다는 트레이드오프가 있다. 요구사항(오후 3시 기준 갱신)에 맞춰 다음과 같이 설계한다.

- **주기**: 매일 15:00 1회 `REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_gmv;` 실행
- `CONCURRENTLY` 옵션을 쓰면 갱신 중에도 기존 데이터로 조회가 계속 가능(락 없이 갱신)하지만, 대상 뷰에 **UNIQUE 인덱스가 반드시 있어야** 하므로 `day` 컬럼에 `CREATE UNIQUE INDEX`를 추가해야 한다.
- **시점 선택 이유**: 오전 중 발생한 당일 주문·취소·환불 처리가 오후 3시경 대부분 반영된다고 가정, 하루 한 번의 갱신으로 "리포트용 데이터 최신성"과 "조회 성능" 사이를 절충한다. 실시간성이 필요 없는 일별/월별 리포트(Q2)이므로 하루 1회 갱신으로 충분하다.
- **자동화 수단**: `pg_cron` 확장을 이용해 `SELECT cron.schedule('refresh_mv_daily_gmv', '0 15 * * *', 'REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_gmv');`로 스케줄링하거나, Airflow 등 외부 스케줄러에서 동일 SQL을 실행하도록 등록한다.
- **한계**: 갱신 주기 동안(최대 24시간) 최신 주문이 리포트에 반영되지 않는다. "당일 실시간 매출"처럼 최신성이 중요한 요구(Q1)에는 MV 대신 인덱스 튜닝(Q1 방식)을 쓰고, "월별 트렌드"처럼 약간의 지연이 허용되는 리포트(Q2)에만 MV를 적용하는 것이 적절하다.

구분	스크린샷
MV 생성 스크립트 실행 화면	mv_create
MV 조회 결과 화면 (`SELECT * FROM mv_daily_gmv ORDER BY day LIMIT 10;`)	mv_select
REFRESH 실행 화면 (선택)	mv_refresh

(위 3개는 아직 캡처가 없어 자리만 비워둔 상태입니다. `report/screenshot/` 폴더에 위 파일명 그대로 저장하면 자동으로 표시됩니다.)

---

4. DBMS 엔진별 옵티마이저 조사 (Postgres / MySQL / Oracle / SQL Server)

항목	PostgreSQL	MySQL (InnoDB)	Oracle	SQL Server
옵티마이저 유형	Cost-Based Optimizer (CBO)	CBO	CBO (통계 기반 + 적응형 최적화)	CBO
통계 수집	`ANALYZE` / autovacuum가 자동 실행	`ANALYZE TABLE`, 8.0+ 히스토그램 지원	`DBMS_STATS` 자동 잡(기본 활성화)	자동 통계 업데이트(기본 활성화)
지원 조인 알고리즘	Nested Loop / Hash Join / Merge Join	주로 Block Nested Loop, 8.0.18+부터 Hash Join 지원	Nested Loop / Hash Join / Sort-Merge Join + 파티션와이즈 조인	Nested Loop / Hash Join / Merge Join
실행계획 확인	`EXPLAIN (ANALYZE, BUFFERS)`, `EXPLAIN (FORMAT JSON)`	`EXPLAIN`, `EXPLAIN ANALYZE` (8.0.18+), `EXPLAIN FORMAT=JSON`	`EXPLAIN PLAN FOR` + `DBMS_XPLAN.DISPLAY`, SQL Trace	`SET STATISTICS PROFILE/IO/TIME ON`, Actual Execution Plan(그래픽)
옵티마이저 힌트	공식 힌트 없음(철학적으로 지양), `pg_hint_plan` 확장으로 우회 가능	`/*+ ... */` 옵티마이저 힌트 공식 지원 (5.7+)	`/*+ ... */` 힌트가 매우 풍부하고 세밀하게 제어 가능	`OPTION (...)` 쿼리 힌트, `LOOP JOIN` 등 조인 힌트 지원
Materialized View	수동/스케줄 `REFRESH` 필요(자동 쿼리 재작성 없음)	공식 미지원(트리거/이벤트 스케줄러로 흉내)	Materialized View + **Query Rewrite** 자동 지원(원본 쿼리를 옵티마이저가 자동으로 MV로 치환)	Indexed View(제약 조건이 까다로움)로 유사 기능 제공
플랜 캐싱	Prepared Statement 캐싱, 5회 이후 제네릭/커스텀 플랜 중 유리한 쪽 자동 선택	8.0부터 쿼리 캐시 제거, 매번 재수립이 기본	Shared Pool에 커서 캐싱, Bind Peeking으로 파라미터 값 기반 최적화	Plan Cache 사용, Parameter Sniffing 이슈 존재(첫 실행 파라미터로 플랜 고정)

본 실습과의 연결점

- 이번 실습은 PostgreSQL 기준으로 진행했지만, "조인 키에 인덱스가 없는 대용량 동등 조인 → Hash Join 선택"이라는 패턴(2절)은 MySQL 8.0.18+/Oracle/SQL Server에서도 유사하게 나타난다.
- 다만 Oracle은 Materialized View에 대해 옵티마이저가 원본 쿼리를 자동으로 MV로 치환해주는 **Query Rewrite** 기능이 있어, Q2처럼 사람이 직접 쿼리를 `mv_daily_gmv` 참조로 바꿔주지 않아도 되는 차이가 있다. PostgreSQL은 이 자동 치환 기능이 없어 애플리케이션/쿼리 단에서 명시적으로 MV를 참조하도록 바꿔야 한다.
- MySQL은 8.0.18 이전까지 Hash Join을 지원하지 않아 `order_items` 같은 대용량 동등 조인에서도 Block Nested Loop을 강제로 써야 했던 버전 차이가 있다 — 즉 동일한 쿼리·데이터라도 MySQL 버전에 따라 이번 실습에서 관측한 것과 다른(더 느린) 조인 전략이 선택될 수 있다.

부록: 전체 튜닝 패턴 요약

패턴	해당 문항
인덱스 컬럼에 연산 적용 금지 → 원본 컬럼 비교 + 복합 인덱스	Q1, Q3
상관 서브쿼리(N+1) → 윈도우 함수로 단일 스캔 전환	Q4, Q6, Q8
반복 조회되는 집계를 Materialized View로 사전 계산	Q2
불필요한 fan-out/중복 스캔 제거 (선집계, 서브쿼리 제거, UNION→CASE)	Q5, Q7, Q9
전체 스캔 후 필터링 → 대상 집합을 먼저 좁힌 뒤 재조회	Q10